

User as Engram: Internalizing Per-User Memory as Local Parametric Edits

Bojie Li
Pine AI

Abstract

Personal memory in a language model is two problems, not one: *content*—the specific facts about a user—and *reasoning skill*—the general ability that turns those facts into answers. The brain keeps the two apart—a sparse, local trace (an *engram*) in the hippocampus for each new episode, a slow neocortex for the shared skills that interpret it—so a new fact need not overwrite everything else. The dominant recipe for LLM personalization, the per-user LoRA adapter, does the opposite: it folds content and skill into one global weight delta, and pays for it. Writing a user’s facts as a per-user LoRA contaminates text that has nothing to do with the user; writing the same facts as local *Engram rows* leaves it mathematically untouched—a roughly $34,000\times$ smaller footprint on unrelated text, by design.

We therefore propose **User as Engram**: store a user’s content as surgical edits to the hash-keyed memory table of an Engram model, and carry the reasoning skill in one *shared* adapter that everyone uses—an artificial analogue of the brain’s complementary learning systems. This layered design matches per-user LoRA’s direct recall while delivering $7.4\times$ higher indirect-reasoning accuracy, and never hurts reasoning relative to the untouched base. Against retrieval, the winner is set by deployment scale, not benchmark size: a per-user Engram table does not grow with the population, so past ~ 100 facts it overtakes a retrieval pipeline running on a $2.5\times$ larger model. We release four Mini-Engram checkpoints—a nano-scale public replication of Engram—all code and data, and a 50-line multi-tenant server with zero cross-user leakage by design.

Code: <https://github.com/19PINE-AI/user-as-engram>

Website: <https://ring0.me/research/user-as-engram>

1 Introduction

Maya has talked to her AI assistant for the better part of a year. Over those months it has learned that her cardiologist is Dr. Elena Vasquez, that Vasquez practises at Lakeside Cardiology, that Maya is severely allergic to penicillin, and that she went vegetarian in 2024. A useful assistant must do three things with this *personal memory*: recall a fact on demand (“*who is my cardiologist?*”), reason over it when the question is indirect (“*I’m visiting my daughter in California next month—where should I go for a heart check-up?*”), and never let one user’s facts leak into another’s. Serving millions of such users from a single transformer [Vaswani et al., 2017] backbone is an open architectural problem.

Human memory already solves a version of it. The brain does not record a new fact by nudging every synapse a little; it writes the episode as a sparse, local trace—an *engram*—in the

hippocampus, while the slow, distributed neocortex supplies the general skills that interpret it [Semon, 1921, Tonegawa et al., 2015, McClelland et al., 1995]. Keeping the fast, local store apart from the slow, shared one is exactly what lets us learn that Maya is allergic to penicillin without disturbing how we reason about allergies in general.

Personal memory in a language model, we argue, has the same two-part structure—*content* (the user’s specific facts) and *reasoning skill* (turning facts into answers)—and each wants a different kind of memory.

Three families of methods address personal memory, and the dominant parametric one conflates these two parts. In-context learning (ICL, Brown et al. 2020, Min et al. 2022) prefixes the facts; retrieval-augmented generation (RAG, Lewis et al. 2020, Gao et al. 2023) fetches them at query time; per-user adapters bake them into the model’s weights.

This parametric family has converged on one recipe—a per-user LoRA [Hu et al., 2022] on a frozen base’s attention projections, fit on each user’s facts (Section 2)—which we adopt as our reference **per-user-LoRA** baseline. A LoRA is one change layered over the whole model: it keeps a user’s facts and the reasoning skill in the same parameters and, like nudging every synapse, changes the model everywhere, not just where the user’s facts are relevant.

Why a per-user LoRA’s edit is global. This is not a tuning artifact—it is what storing a fact in a shared function costs. A LoRA has no address: to raise the probability of one fact, gradient descent must bend $Q/K/V/O$ along whatever directions most cheaply fit it, and those directions are active on unrelated text too, so every forward pass sees the edit.

We see this as two effects that come apart. The *architectural* cost—extra loss on unrelated text—is a property of the parameterization: large, and present on every base we tried. The *user-visible* cost—worse reasoning—is a property of the base’s robustness: a heavily instruction-tuned model absorbs the perturbation, a fragile base LM does not. Reading only the second signal on robust bases is what let prior work call the recipe harmless; the first shows the edit was never local (Section 3).

The fix: put each fact where storage belongs. We move the content back onto a storage substrate. **User as Engram** writes each of Maya’s facts as a surgical edit to the hash-keyed memory table of an Engram-pretrained model [Cheng et al., 2026], whose gated lookups fire only on suffix N-grams via deterministic multiplicative-XOR hashing, while the reasoning skill that interprets the facts lives in one *shared* adapter trained across other users.

Why it works: the table is content-addressable and inert off its key, so writing a fact changes the model’s output only at the positions whose trigger N-gram addresses it—and we can verify the change is *exactly* 0.000 everywhere else, through every subsequent attention and MLP layer, not merely small in the limit (Section 4.3, Figure 6).

Why the table is the right place for personal memory, in three ways the weights cannot offer: writes are isolated, so per-user facts compose and never leak across tenants by construction; the index matches the access pattern, since personal facts are queried by their surface form and a suffix-N-gram hash is a natural key for exactly that; and a row is a fixed ~ 88 KB independent of the model, with a deterministic address that can be swapped in per request, so storage grows with the number of users, not the size of the model.

Contributions.

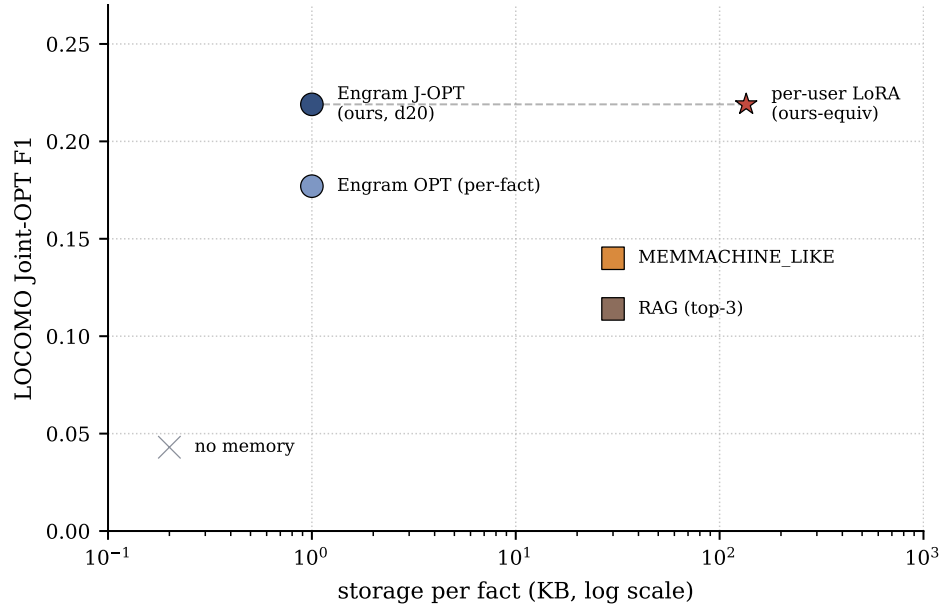


Figure 1. Cost-quality trade-off across personal-memory methods at the same answering LM (Mini-Engram-d20@1536). User-as-Engram Joint OPT matches a per-user LoRA’s LOCOMO F1 at $161\times$ smaller per-user storage (88 KB vs. 14.2 MB at 100 facts). Retrieval baselines sit on the steep low-storage side of the curve but cap out below Engram on quality.

1. **The cost of storing facts in shared weights** (Section 3). A per-user LoRA has no address: to fit even one fact, gradient descent must bend the same $Q/K/V/O$ directions that are active on unrelated text, so the edit changes every forward pass—it contaminates globally. An Engram row, by contrast, fires only on its trigger N-gram. On a controlled base this is a $\sim 34,000\times$ difference in extra loss on unrelated text—an architectural property, not a tuning artifact. We separate this robust architectural cost from the variable, base-dependent effect on user-visible reasoning.
2. **User as Engram** (Section 4): a way to write a user’s facts as surgical, local edits to an Engram model’s memory table, with a verified locality property—*exactly* zero change at every non-trigger position. We release four Mini-Engram checkpoints (178 M–1.22 B; a nano-scale public replication of Engram), all code and data, and a 50-line multi-tenant server with zero cross-user leakage.
3. **A layered design** (Section 6) that pairs per-user Engram content with one shared reasoning adapter—an artificial complementary learning system—and beats every all-in-one baseline on direct recall, indirect reasoning ($7.4\times$ over per-user LoRA), contamination, and per-user reasoning regressions. The lead survives a personal $\rightarrow$ medical schema shift.
4. **A deployment-scale comparison with retrieval** (Sections 6.4–6.6): which method wins is set by facts-per-user and population size, not benchmark size. Because a per-user Engram table does not grow with the population, it overtakes a strong retrieval pipeline—running on a $2.5\times$ larger model—once the knowledge base passes ~ 100 facts.

2 Background

The Engram architecture. Cheng et al. [2026], part of the DeepSeek line of sparse models [Dai et al., 2024, DeepSeek-AI, 2024], introduce *conditional memory* as a form of sparsity that complements mixture-of-experts (MoE, Shazeer et al. 2017, Dai et al. 2024). At each token position t , suffix N-grams of canonicalised tokens are hashed via K multiplicative-XOR heads into prime-sized embedding tables $E_{n,k}$. Retrieved rows e_t are projected by learned W_K, W_V and gated by an attention-style scalar $\alpha_t = \sigma(\text{RMS}(h_t)^\top \text{RMS}(W_K e_t) / \sqrt{d})$. The output $\alpha_t W_V e_t$ is added to the residual stream at the selected Engram layer. Crucially, the address $z_{t,n,k}$ is deterministic from the input token IDs—known before the forward pass—so the table can be offloaded to host DRAM without GPU contention.

LoRA-as-memory family. Per-user/per-document LoRA adapters [Su et al., 2025, Tan et al., 2024, Zhuang et al., 2024, Charakorn et al., 2025, Tan et al., 2024b, Bini et al., 2025] train a low-rank delta [Hu et al., 2022, Houlsby et al., 2019, Mangrulkar et al., 2022]—one member of a broad parameter-efficient fine-tuning family that also includes prefix-tuning [Li & Liang, 2021] and adaptive-rank and quantized variants [Zhang et al., 2023, Dettmers et al., 2023]—on $Q/K/V$ (and sometimes MLP) matrices on top of a frozen base. It is fit per user via NTP on a (observation, fact, QA) mixture, via knowledge distillation from a teacher (DyPRAG [Tan et al., 2025], DistilledPRAG [Chen et al., 2025]), or via hypernetwork emission (T2L [Charakorn et al., 2025]). All variants *edit the model weights globally*: every forward pass sees the LoRA’s change.

Memory architectures and adjacent work. Engram [Cheng et al., 2026] sits in a lineage of trainable key–value memory modules—a view foreshadowed by the finding that ordinary feed-forward layers already act as key–value stores [Geva et al., 2021]: PKM [Lample et al., 2019], PEER [He, 2024], Memory Layers at Scale [Berges et al., 2025], Ultra-Mem [Huang et al., 2024b], OverEncoding [Huang et al., 2025], SCONE [Yu et al., 2025], BLT [Pagnoni et al., 2025], and SuperBPE [Liu et al., 2025]. None addresses surgical per-user insertion at inference. The closest analogue in the diffusion-model world is LoRA stacking for Stable Diffusion [CivitAI, 2024]; in the LLM world, LoRAHub [Huang et al., 2023] trains a combiner network. Engram override maps with disjoint addresses are *additive without a combiner* (Section 5). A complementary line keeps memory *external* but makes it executable: User as Code [Li, 2026] compiles a user’s history into typed Python state plus deterministic rules, trading our in-weights locality for an interpreter that resolves contradictions and aggregates exactly. The two are mirror images—in-weights edits on one side, external executable state on the other—and we contrast them in Section 5.

Memory systems for LLM agents. A fast-growing body of work gives an agent long-term memory by keeping facts *outside* the weights and retrieving them at query time. MemGPT [Packer et al., 2023] pages memories in and out like an operating system; Mem0 [Chhikara et al., 2025], A-MEM [Xu et al., 2025], Zep [Rasmussen et al., 2025], MemOS [Li et al., 2025], EverMemOS [Hu et al., 2026], and MemMachine [Wang et al., 2026] add extraction, consolidation, and graph- or profile-structured stores; and a reinforcement-learning line learns *how* to write and read that store—Memory-R1 [Yan et al., 2025], AgeMem [Yu et al., 2026], and Mem- α [Wang et al., 2025]. Recent surveys catalogue the design space [Zhang et al., 2024, Wu et al., 2025, Du, 2026]. All of these keep memory in natural-language or graph form and consult it by retrieval;

none writes a fact *into* the model. The name clash is worth flagging explicitly: the concurrent ENGRAM system [Patel & Patel, 2025] shares our term but is its opposite—it orchestrates memories in natural language, non-parametrically, whereas our *Engram* is a parametric edit to the model’s own hash-keyed memory table. We use Mem0- and MemMachine-style retrieval as our strongest non-parametric baselines (Section 5.7) and ask the distinct question of *when* writing a fact into the weights beats keeping it in a retrievable store—a trade-off that recent cost analyses also weigh for long-context agents [Pollertlam & Kornsuwannawit, 2026].

Personalizing language models. Adapting a model to an individual user is an active area in its own right [Zhang et al., 2024, Liu et al., 2025, Xu et al., 2026], with benchmarks such as LaMP [Salemi et al., 2024] measuring personalized generation. Most methods personalize through the prompt or a retrieval store; the per-user LoRA line above is the parametric alternative, and User as Engram is our attempt to win the locality of a prompt at the zero-context cost of a weight edit.

Retrieval and non-parametric memory. Retrieval-augmented generation [Lewis et al., 2020, Gao et al., 2023] is the dominant non-parametric route, from early retrieve-then-read pre-training [Guu et al., 2020, Khandelwal et al., 2020] through large-scale retrieval pre-training [Borgeaud et al., 2022, Izacard et al., 2023] to self-reflective retrieval [Asai et al., 2024]. Its quality is bounded by the encoder’s ability to surface the right evidence among many candidates—exactly the failure mode we measure as the knowledge base grows (Section 6.6).

Personal memory benchmarks. Long-term personal memory is evaluated by LOCOMO [Maharana et al., 2024], LongMemEval [Wu et al., 2024], BEAM [Tavakoli et al., 2025] (multi-turn evolving beliefs), and PersonaMem-v2 [Jiang et al., 2025]. We use synthetic per-user fact corpora for the within-user fact-density sweeps (tighter control over the parameter we vary), and LOCOMO (Section 5.8) for end-to-end evaluation against retrieval baselines. We chose LOCOMO over LongMemEval and BEAM because its per-question evidence pointers let us train a per-fact Engram row and a retrieval entry against the same gold sentence—an apples-to-apples “same gold, different method” comparison. LongMemEval and BEAM remain open for future work.

Knowledge editing and recitation. Knowledge editing changes a stored fact in place: locate-and-edit methods (ROME [Meng et al., 2022], MEMIT [Meng et al., 2023]) write FFN weights at causal-traced locations, hypernetwork editors (MEND [Mitchell et al., 2022a]) and external-memory editors (SERAC [Mitchell et al., 2022b], WISE [Wang et al., 2024]) route around the base weights, and work on knowledge neurons [Dai et al., 2022] and editing surveys [Yao et al., 2023] maps where facts live. Ripple-effect benchmarks (MQuAKE [Cohen et al., 2023], RippleEdits [Cohen et al., 2024], CounterFact [Meng et al., 2022b]) measure their indirect consequences. Unlike these methods, which overwrite knowledge the model was pretrained with, we *add* per-user facts the model never saw, and care about leaving its other behaviour untouched. Recitation methods (SR-RAG [Wu et al., 2025], recitation-augmented generation [Sun et al., 2023]) train or prompt the model to surface stored facts before reasoning. We apply the recitation idea to per-user adapters in Section 3, and treat it as a candidate mitigation for Engram in our discussion.

Complementary learning systems and continual learning. The extra loss a per-user LoRA imposes on unrelated text is a modern instance of *catastrophic interference*: training on something new overwrites what was there before [McCloskey & Cohen, 1989], which decades of continual-learning work fight by protecting important weights [Kirkpatrick et al., 2017] or replaying old data, with the problem persisting for large language models [Shi et al., 2024]. The brain avoids it with a different architecture, and the word *engram* points to how: an engram is the physical trace a memory leaves in neural tissue [Semon, 1921], and *engram cells*—sparse neuron populations whose reactivation triggers recall—sit largely in the hippocampus [Tonegawa et al., 2015], while general skills live in the slow, distributed neocortex. Complementary learning systems theory [McClelland et al., 1995, Kumaran et al., 2016] argues this split is what lets a new episode be written without overwriting old skills. User as Engram is the same division made architectural: the per-user Engram row is the fast, sparse, local trace (the hippocampal engram); the shared LoRA and frozen backbone are the slow, distributed skill (the neocortex); the gated trigger-N-gram lookup completes a memory from a partial cue; and our measured locality ($\Delta\text{bpb} +0.00005$ on unrelated text, Section 3) is the engineering form of the pattern separation that keeps the two from interfering. Where continual learning asks one set of weights to absorb new facts gracefully, we sidestep the interference by writing each fact to its own address and leaving the shared weights untouched.

3 Per-user LoRA contaminates globally

Weights are an excellent place to keep knowledge—pretraining packs millions of facts into them accurately. They are a poor place to keep *one user’s* facts written as a *per-user weight edit*, and not for lack of capacity. A per-user LoRA edits $Q/K/V/O$ in every layer, so its edit is unconditioned (part of every forward pass) and must be fit per user from a handful of examples. Two problems follow, and we measure both: the edit is global, so it contaminates text that has nothing to do with the user; and the facts, once trained in, are *recalled but not readily reasoned over*. The first is an architectural cost paid on every base; the second matters most for personalization, and is why a fact belongs in a store the shared reasoning can read rather than in a per-user copy of the weights.

3.1 Architectural contamination on the same base (Mini-Engram-d20)

We hold the base fixed (Mini-Engram-d20@1536, 1.22 B parameters) and train a per-user LoRA (rank-64 on $Q/K/V$, 1,500 steps per fact, single-token gold) on the synthetic user fact sets used throughout. For each of 20 test users we measure val_bpb on a held-out 262 K-token ClimbMix shard—text unrelated to the user’s facts—before and after the edit. Per-user Engram-row Joint OPT is measured under the identical protocol on the same base.

The figure shows the gap; the mechanism is the point. A LoRA cannot write a fact without bending a function every input shares—to lift the probability of Maya’s cardiologist it moves $Q/K/V/O$ along directions that also fire on unrelated text—so the loss on held-out text rises (more than *tripling* here), and it rises a little further with each fact added, because every new fact bends the function again. An Engram row is written to an address instead: it is read only when its trigger N-gram is hashed, and leaves the loss on everything else unchanged to four decimal places. This is the difference between editing a function and writing to a store, and it is precisely what makes co-locating many users feasible in one and hopeless in the other:

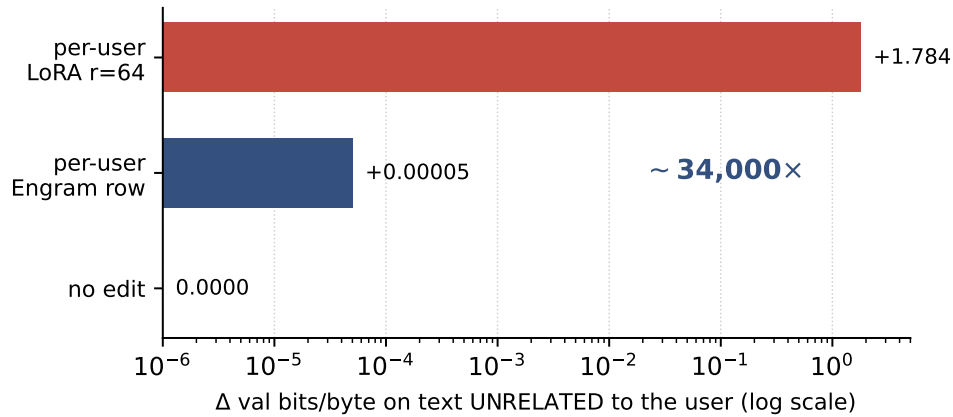


Figure 2. Architectural contamination on held-out text, Mini-Engram-d20 (canonical seed S0, $n=20$ users). A per-user LoRA more than triples val bpb on text *unrelated* to the user’s facts (+1.784; per-user range [+0.44, +3.74]; 17/20 users worse), while a per-user Engram row leaves it unchanged to four decimals (+0.00005; 0/20 worse)— $\sim 34,000\times$ less, by design. The 3-seed mean ratio is $\sim 33,000\times$ (Appendix O).

contamination from a shared delta accumulates and cannot be quarantined per user, whereas addressed writes have zero cross-talk by construction (Section 7).

3.2 Recall without reasoning

A per-user LoRA learns to *recall* its facts—direct recall is near-perfect—but the harder half of personal memory is *reasoning over* them, and that is where it struggles. The reason is composability. Knowledge acquired in pretraining is laid down in a form the model’s reasoning can chain; a per-user adapter fit to a handful of examples instead memorizes a flat trigger→answer mapping that the shared reasoning does not readily pick up and combine. To answer an indirect question the adapter would have to carry the reasoning skill as well, learned from the same few examples—hard to train, and it does not transfer to facts it never saw. This recall-without-reasoning gap is the recurring finding of the per-user-adapter line [Su et al., 2025, Tan et al., 2024, Zhuang et al., 2024]: the facts go in, but indirect questions over them do not reliably come out.

What a user sees in addition depends on the base. We replicate the per-user LoRA recipe (rank-64 NTP on observation/fact/QA mixtures, 12 epochs $\times$ 200 samples) on three instruction-tuned bases—Qwen2.5-7B [Qwen Team, 2025a], Llama-3.1-8B [Grattafiori et al., 2024], and Mistral-7B [Jiang et al., 2023]—and one base LM (Figure 3):

The split is clean (Figure 3). On a base LM the global perturbation also disrupts fragile completion behaviour, so indirect recall drops outright; on an instruction-tuned base the base’s own reasoning carries the indirect question and masks the adapter’s inability to provide it. The caution is methodological: judging a per-user LoRA by task accuracy on a strong instruction-tuned base hides both the contamination and the fact that the adapter never learned to reason over what it stored. The reasoning has to come from somewhere—and Section 6 supplies it from one shared skill instead of asking each per-user edit to relearn it.

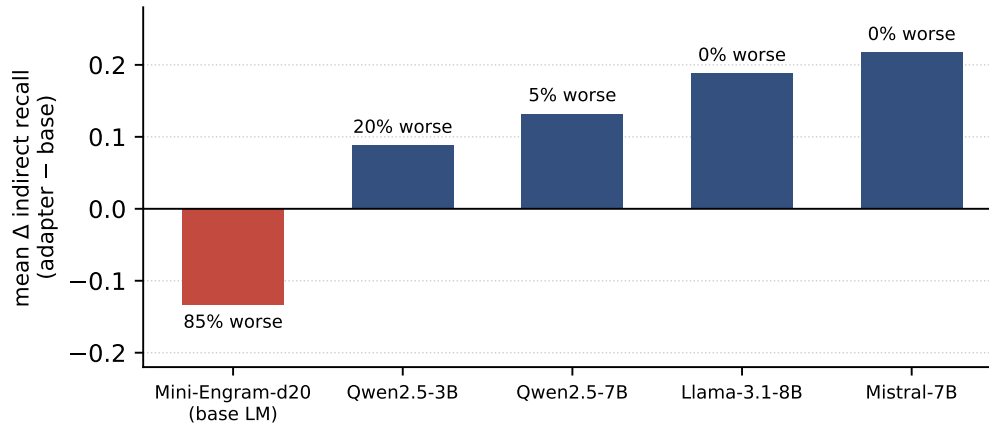


Figure 3. Cross-base LoRA scaling: mean change in indirect recall (adapter – base) with the fraction of users it hurts. On the base LM (Mini-Engram-d20) a per-user LoRA disrupts fragile completion behaviour (85% worse, $\Delta = -0.133$); on every instruction-tuned base the reasoning skill absorbs the perturbation (0–20% worse, Δ from +0.088 to +0.217).

4 Method: User as Engram

We instantiate the “local per-user edit” concretely: surgically write per-user fact rows into the hash-keyed memory table of an Engram-pretrained model. Three pictures carry the method—where that table sits in the network (Figure 4), how a fact is written into it (Figure 5), and how the writes are served per user (Figure 7)—and the subsections below walk through each.

4.1 Where the Engram lives

Most of an Engram model is an ordinary transformer [Vaswani et al., 2017]; the only addition is a content-addressable lookup spliced into a few layers (Figure 4; two layers in our Mini-Engrams). At every position, the recent tokens—a suffix N -gram—are hashed by K deterministic multiplicative-XOR heads into addresses in a large embedding table E . The retrieved rows are projected by W_K, W_V , weighted by an attention-style gate $\alpha = \sigma(\cdot)$, and added to the residual stream, exactly where attention and the MLP also write. The lookup is thus a read from memory, but keyed by the surface form of the recent tokens rather than by query-key similarity across the sequence.

Two properties make this a genuine store rather than just more parameters, and both are what we exploit. First, the address is a deterministic function of the input token IDs—known *before* the forward pass—so we know exactly which rows a given query will read, and the table can be offloaded to host DRAM. Second, the gate makes the read sparse and conditional: most positions retrieve essentially nothing, and a stored row affects the output only at positions whose N -gram addresses it. This is the addressable, sparse store that per-user facts want—the opposite of the global weight edit of Section 3. Attention and the MLP are dense and global—every position, every weight—whereas the Engram read touches a handful of rows at a handful of positions, so editing one row changes the model only where that row is addressed. The rest of this section turns that into a per-user memory.

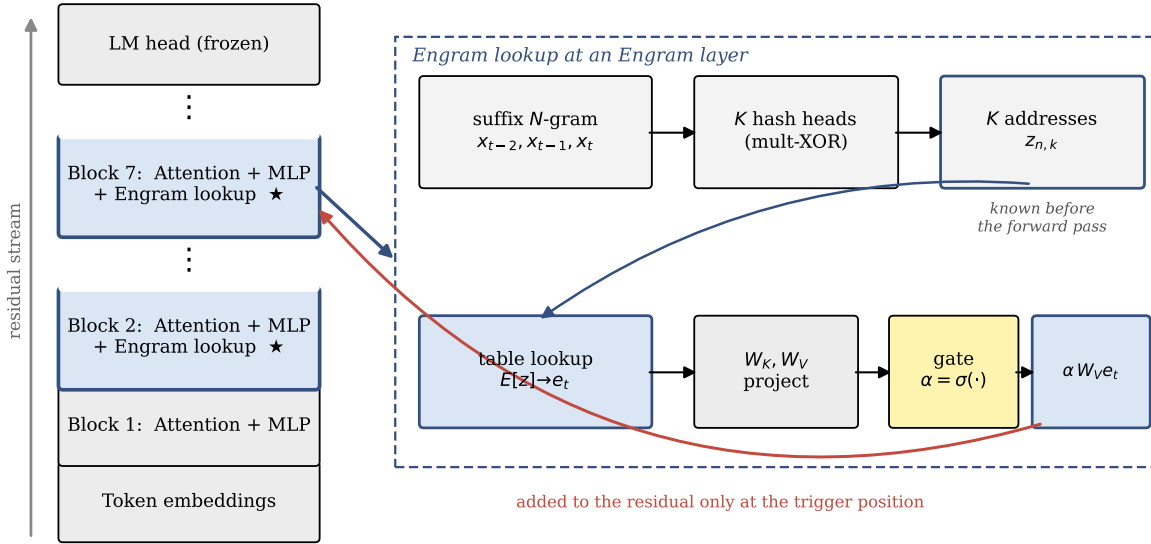


Figure 4. Where the Engram lives in the transformer. The backbone is a standard transformer; at a few designated Engram layers (★) a content-addressable lookup runs alongside attention and the MLP. At a position, the recent tokens’ suffix N -gram is hashed by K heads into K table addresses (deterministic, hence known before the forward pass); the retrieved rows are projected by W_K, W_V , gated by α , and added to the residual stream—but the gate fires only where the trigger N -gram is present. Unlike attention and the MLP, which touch every position and every weight, the Engram read is sparse and addressable; that is what lets a single written row change behaviour locally.

4.2 Surgical row insertion

To write a fact, we put its answer where its trigger is read. The trigger’s suffix N -gram hashes to a fixed, sparse set of table rows R_f —about 16 rows for our $N=3, K=8$ configuration—deterministically, touching no other row. What remains is *what* to write into those rows, for which we use three strategies of increasing cost and fidelity (per-strategy recall against a RANDOM control in Appendix D):

- **UNEMBED_P** (closed form): solve in one step for the row value whose projection steers the next-token distribution toward the answer—a single matrix–vector product, <1 ms, no training.
- **OPT**: refine that value with a few gradient steps per fact (~ 1 s), trading time for accuracy.
- **Joint OPT** (our default beyond ~ 30 facts/user): optimise *all* of a user’s rows together rather than one fact at a time.

Joint OPT is the default because of the one place this design has to earn its keep. When many of a user’s facts are live at once, their rows are read in the same forward passes and interfere; optimising them jointly lets the rows settle into values that coexist, which is what holds recall up as facts-per-user grows (Section 5). Crucially, every write still lands only in that user’s R_f , so however many facts or domains are stacked, the store stays additive and private—the locality is the architecture’s, not a constraint we have to impose.

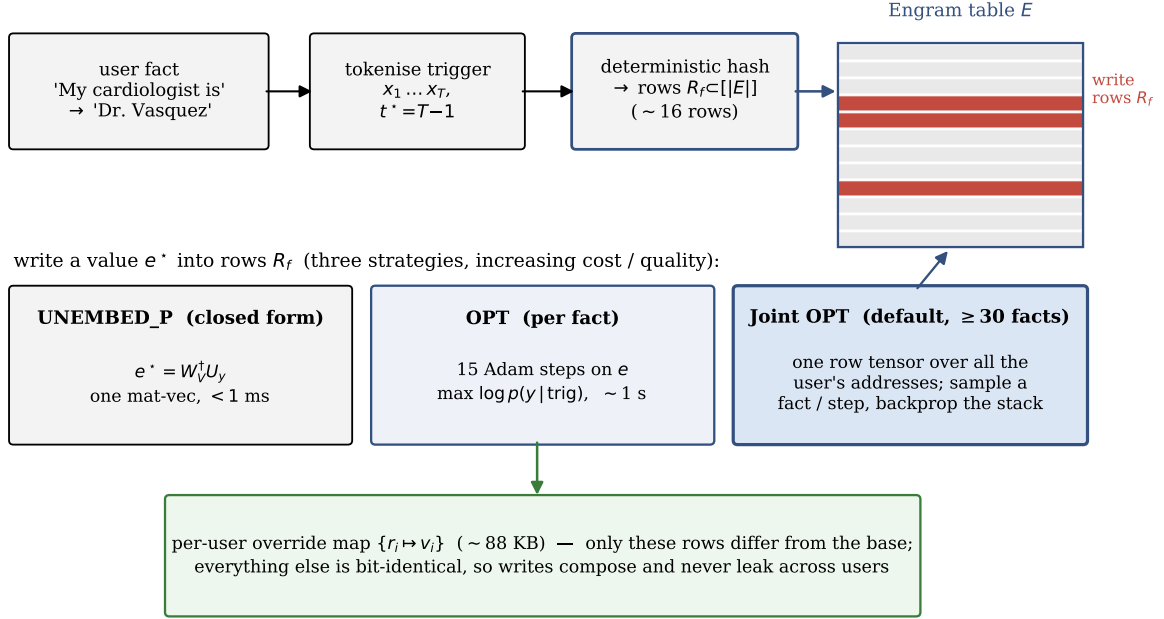


Figure 5. Inserting a user fact into the Engram. A fact is reduced to *where* and *what*: the trigger’s suffix N -gram hashes to a sparse set of row addresses R_f (*where*), and one of three strategies writes a value e^* into those rows (*what*). UNEMBED_P solves for e^* in closed form; OPT takes a few gradient steps per fact; Joint OPT—our default beyond ~ 30 facts/user—optimises all of the user’s rows together so they do not interfere. The result is a per-user override map of ~ 88 KB; every row outside R_f , and the entire backbone, is left bit-identical.

Read together, the two figures give the method in one sentence: **remembering a fact is writing a few rows**. A fact decomposes into an address set R_f (*where*, fixed by the trigger’s hash) and a value e^* (*what*, set by one of the strategies above), and the write touches nothing else. The locality we rely on is therefore *inherited from the architecture*—the read was already sparse and addressed (Section 4.1)—rather than enforced by a separate mechanism. That is why per-user and per-domain writes compose without a combiner and never leak across users (Section 5), and Section 4.3 verifies that “touches nothing else” is literally exact.

4.3 Local edit verified

Writing a UNEMBED_P marker at the last Engram layer and measuring $\|x_{\text{after}}^{(\ell)} - x_{\text{before}}^{(\ell)}\|$ at every layer and every position yields the property by inspection (Figure 6): the diff is *exactly* 0.000 at every non-trigger position through 5 subsequent attention/MLP layers, while the trigger-position diff is 123.0 at layer 8 and 1.54 at the final layer. **This is the architectural property that distinguishes Engram from LoRA**: no contamination of non-trigger forwards. Appendix B consolidates this locality result with the read/write probe, the LogitLens depth analysis, and the multi-hop decomposition into an end-to-end account of how an inserted fact is addressed, stored, and retrieved.

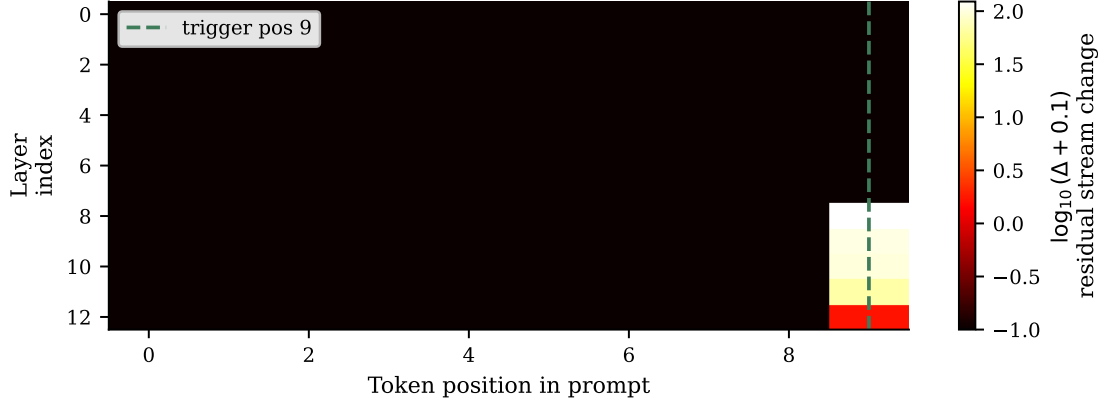


Figure 6. Engram surgical insertion is spatially perfect. Heatmap of $\|x_{\text{after}}^{(\ell)} - x_{\text{before}}^{(\ell)}\|$ for a UNEMBED_P insertion at the last Engram layer ($L_{\text{eng}}=7$, Mini-Engram-d12). The diff is exactly 0.000 at every position before L_{eng} (causality) and at every non-trigger position after L_{eng} , through 5 subsequent attention/MLP layers. Compare with LoRA, where every position in every forward sees the global delta.

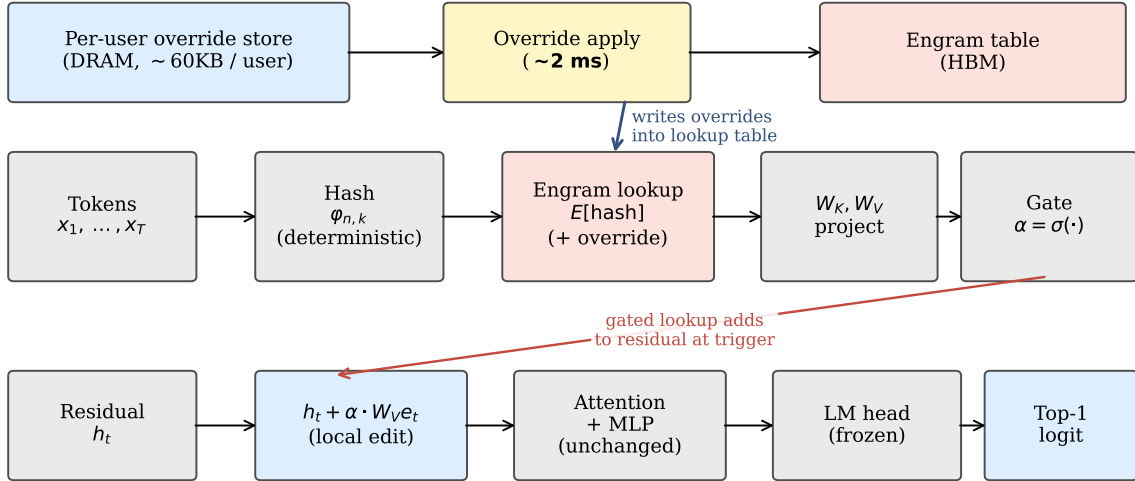


Figure 7. EngramServer architecture. Per user, an override map of (row_idx, row_vector) pairs lives in DRAM. On each request, the server saves the originals at the affected addresses (~ 2 ms), writes the user’s overrides, runs the forward pass, then restores. The Engram lookup at the configured layer transparently sees the override values; the gate fires only at the trigger N-gram. There is no router and no graph rewrite.

4.4 Per-user override tables and additive composition

The production design uses per-user override tables. Each user has a small dictionary $\{r_i \mapsto v_i\}$ of fact rows. At request time, the server saves the originals at the affected addresses, writes the user’s overrides, runs the forward, and restores. Cross-user leakage is *zero by design*—user A’s overrides are not in the table when user B queries.

Override maps with disjoint addresses commute, so corporate facts + user facts (or any number of domain-specific Engrams) **stack additively at inference** without retraining a combiner. This mirrors how Stable Diffusion LoRAs additively stack, but at the row level. Section 5

Table 1. Mini-Engram pretraining at the ablation-optimal recipe. All use the large Engram ($50\text{ K} \times 256 = 51.2\text{ M}$ Engram params). Trained on a single NVIDIA RTX PRO 6000 Blackwell (102 GB) in bf16.

	d8 v2	d12 v2	d12@1280 opt.	d20@1536 opt.
depth $\times$ width	8×512	12×768	12×1280	20×1536
Engram layers	2, 5	2, 7	2, 7	2, 11
total params	178 M	339 M	625 M	1.22 B
scaling-params	42 M	110 M	278 M	617 M
ClimbMix tokens (12 t/p)	0.50 B	1.32 B	3.34 B	7.40 B
final val_bpb	0.912	0.827	0.770	0.730
wall-clock (single GPU)	~ 50 min	~ 5 h	~ 10.5 h	~ 70 h (shared GPU)

confirms that additive composition is lossless when the domains have disjoint trigger templates.

5 Experiments

This section reports five blocks of measurements, each answering a distinct question about User-as-Engram: **(a) per-fact recall and density** (Sections 5.2–5.3): how accurately do single and multiple per-user facts surface? **(b) cost** (Sections 5.4–5.5): storage and wall time vs. per-user LoRA. **(c) direct comparison with memory systems / RAG / LOCOMO** (Sections 5.7–5.8): how does our method compare against retrieval baselines on the same answering LM? **(d) scaling ablations** (Sections P.1–P.3): capacity vs. token budget vs. dense size. **(e) probes beyond direct recall** (Sections 5.9–5.10): multi-hop, free-continuation, paraphrase. We use three corpora throughout: the **base** 200-fact benchmark (100 USER + 100 ORG facts), the **XL** corpus of 1,000 USER + 1,000 ORG templated facts (Section P.2), and the **XXL** corpus of 3,132 distinct trigger templates (used for high-density and distinct-template stress tests).

5.1 Setup: Mini-Engram substrate

Before measuring per-user insertion we need an Engram backbone to insert into. Cheng et al. [2026] released only architectural reference code; no Engram-trained weights are public. We graft Engram into Karpathy’s nanochat [Karpathy, 2026]—a GPT-2-style backbone [Radford et al., 2019] optimised with Muon+AdamW [Jordan et al., 2024]—and train four Mini-Engram models on a single Blackwell GPU at the ablation-optimal recipe (Section P.1): the same large Engram table ($50\text{ K} \times 256$, 51.2 M params), trained at Karpathy’s 12 tokens-per-param (t/p) budget on the scaling-params for every dense size.

Model-name notation. Throughout the paper we use $d\{\text{depth}\}@ \{\text{width}\}$ for the ablation-optimal Mini-Engrams (d12@768, d12@1280, d20@1536) and v1/v2 to denote earlier and final training runs at the same shape. Where the width is omitted (e.g. d12 without a suffix), we mean the v2 (ablation-optimal at width 768, 339 M total) checkpoint. The four headline checkpoints, their param counts and token budgets are in Table 1; all four share the same 51.2 M Engram table.

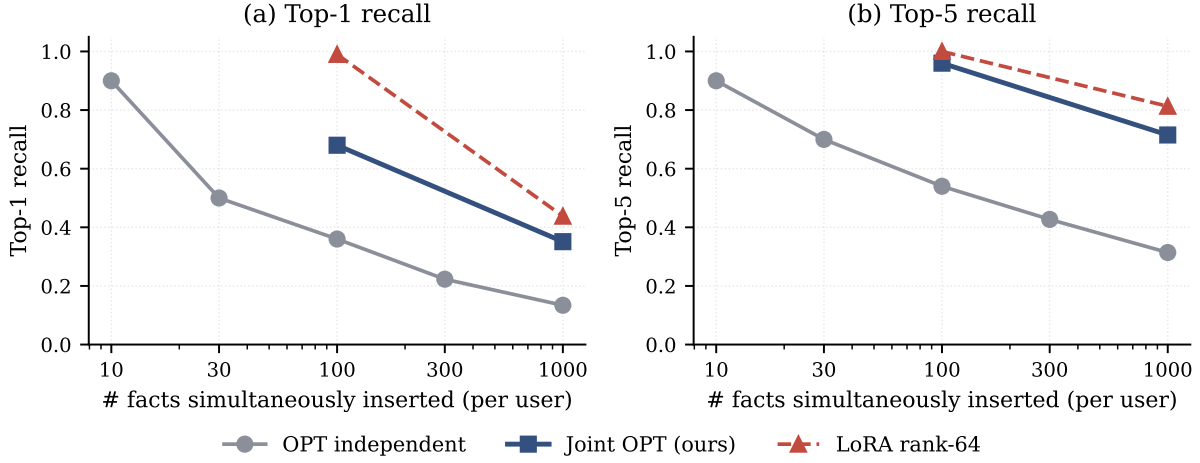


Figure 8. Within-user fact-density: top-1 (left) and top-5 (right) recall vs. number of facts inserted simultaneously into one user’s override map. Joint OPT (blue) closes most of the gap to LoRA rank-64 (red dashed) at $161\times$ less storage (88 KB vs. 14.2 MB at 100 facts).

We reproduce Engram’s two signature findings. Suppressing the Engram lookup hurts factual recall far more than reading comprehension—the factual-vs-reading sensitivity asymmetry of Cheng et al. [2026] (their §6.3), at the $\sim 10\times$ smaller magnitude expected for a model two orders smaller on a corpus three orders smaller. The Engram layers also act as effective extra depth: a LogitLens probe shows our small Engram model resolving its prediction earlier than its dense-only twin, matching their §6.1 (Appendix Figure 23).

5.2 Single-fact recall

A single inserted fact surfaces within ~ 8 points of the in-context ceiling, at three orders of magnitude lower per-user cost than training a full LoRA (per-strategy recall against a RANDOM control in Appendix D).

5.3 Within-user fact density and Joint OPT

The harder question is what happens when many facts are live at once: the rows in a single user’s override table interfere during the forward pass. We sweep that density curve on Mini-Engram-d12 (Figure 8).

Joint OPT is the recommended default for ≥ 30 facts/user: it tracks multi-fact LoRA rank-64 to within a few top-5 points while keeping top-5 recall above 90% out to ~ 300 facts/user, all at up to $\sim 161\times$ less per-user storage (Figure 8; per- n recall and storage in Appendix P).

The density ceiling is set by the Engram table, not the dense backbone. Holding Engram capacity fixed and scaling the dense backbone does *not* lift recall at 1000 facts—it slightly degrades it. The limit is therefore gradient interference among the co-active rows in one user’s table, not backbone capacity, which is why the multi-fact-in-the-loss recipe change (Section 8.3)—rather than further dense scaling—is the promising path to breaking it.

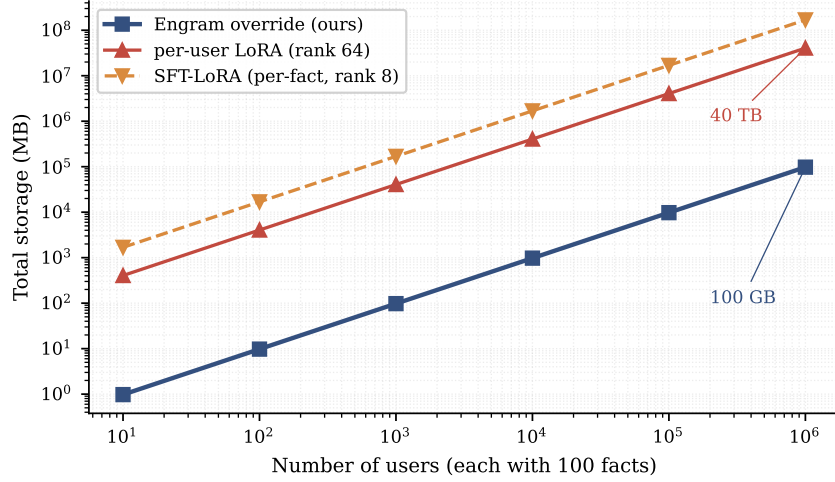


Figure 9. Per-user storage scales linearly with user count. Engram override is $\sim 161\times$ smaller at 100 facts/user (88 KB vs. 14.2 MB) and the gap widens to $\sim 1700\times$ at 10 facts/user (LoRA’s parameter count is fact-count-independent while Engram grows linearly). At 1 M users $\times$ 100 facts/user, Engram storage is 100 GB vs. per-user LoRA’s 40 TB.

5.4 Storage scaling at deployment

At realistic deployment scales (10^4 – 10^6 users) the $\sim 161\times$ ratio is the difference between fitting on one server vs. requiring distributed storage.

5.5 Method comparison: ICL, RAG, SFT-LoRA, per-user LoRA

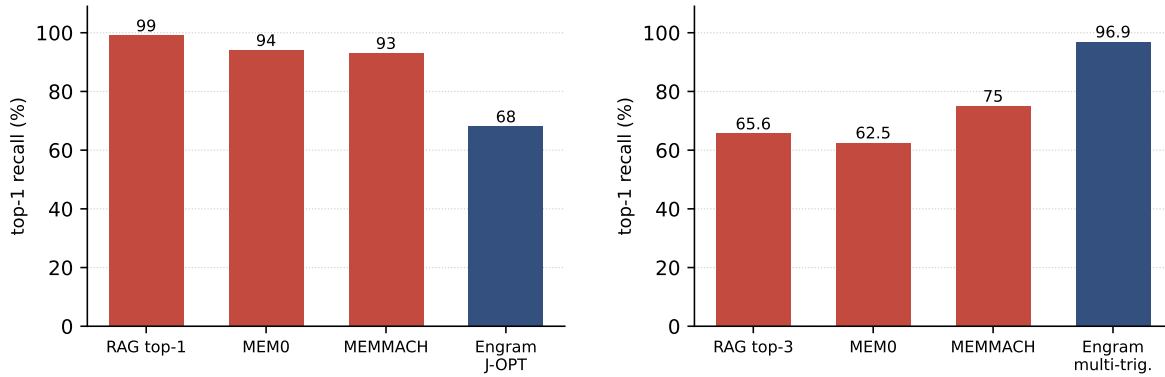
Figure 1 plots these methods on a cost-quality trade-off where per-user storage spans four orders of magnitude. The lesson is that LoRA’s storage premium buys headroom that the rank-8 ceiling has already spent, not asymptotic recall: a single fact matches the in-context ceiling at a tiny fraction of LoRA’s cost, and at high density Joint OPT tracks even rank-64 LoRA at tens-of-times less storage—the gap actually inverting in Engram’s favour by 1 000 facts.

5.6 Multi-domain additive composition

Because two override maps with disjoint addresses commute, corporate and user Engrams stack additively at inference without retraining a combiner: each domain keeps its alone-recall under composition. The qualifier is template overlap—when two domains *share* trigger templates their addresses collide and the later-applied override wins. This gives a clean architectural rule: **additive composition for disjoint-template domains; per-user override tables for same-template tenants** (measurements in Appendix F).

5.7 Comparison against memory systems

We compare User-as-Engram against state-of-the-art memory systems that target the same use case (per-user / personal memory). All systems use the *same* Mini-Engram-d12 as the final-answer LM; the only difference is how facts are stored and retrieved. Our MEM0 and MEMMACHINE baselines follow the retrieval recipes of Mem0 [Chhikara et al., 2025] and MemMachine [Wang et al., 2026]. The sentence encoder used by the RAG / MEM0 / MEMMACHINE



(a) Retrieval wins on exact-trigger queries—but Engram pays *zero* context tokens (0 vs. 16–63) for its 68%.

(b) Engram multi-trigger insertion wins on paraphrases by 22 points, again at zero context.

Figure 10. Direct recall vs. memory systems, all sharing the same Mini-Engram-d12 answering LM (100 USER facts, XXL corpus). **a** When the query is the fact’s stored prefix, nearest-neighbour retrieval is near-perfect and beats Engram’s 68% top-1—but at a context-token cost Engram does not pay. **b** When the query is a paraphrase, retrieval drops to 60–75% as the encoder confuses surface forms, while multi-trigger Engram insertion (writing at every anticipated paraphrase, ~ 5 s/fact) reaches 96.9% top-1. This holds at both sizes we tried, d12@1280 and d20@1536.

baselines is all-MiniLM-L6-v2 [Wang et al., 2020, Reimers & Gurevych, 2019] (80 M parameters; comparable in size to a production retriever).

This subsection probes *direct* fact recall under trigger-exact and paraphrased queries; the separate *indirect-reasoning* comparison (does retrieval recover the gold fact across hops, and what happens as the KB grows?) is deferred to Sections 6.4–6.6.

Setup. 100 USER facts (XXL corpus). Two query distributions:

- **Trigger-exact** (Figure 10, left): the query is the fact’s stored prompt (e.g. “My favorite spice is”). Easy retrieval setting—the query prefix matches the fact verbatim.
- **Paraphrased** (Figure 10, right): the query is a different surface form (e.g. “I love the spice”, “My preferred spice is”) of the same underlying fact: 16 facts $\times$ 4 paraphrase queries each.

The story flips on paraphrased queries. When the query is reworded, the sentence encoder confuses surface forms and retrieval slips, while a single-trigger Engram misses the reworded N-gram entirely. The fix is to write a row at every anticipated paraphrase: multi-trigger insertion then wins on paraphrase by a clear margin at zero context cost (Figure 10), because the contract is explicit—the trigger N-gram—rather than left to an encoder’s notion of similarity. This verdict (*retrieval wins on the verbatim prefix; multi-trigger Engram wins under paraphrase*) is invariant to dense scale: the Engram numbers barely move across our three sizes, and only the retrieval baselines drift as their answering-LM improves (Appendix P).

Limitations of this comparison. Our MEMMACHINE_LIKE baseline is the retrieval primitive (top-3 sentence-encoder retrieval over atomic facts), not the full MemMachine system on conversational episodes (LOCOMO [Maharana et al., 2024], LongMemEval [Wu et al., 2024]); the canonical MEMO evaluation includes an LLM-based fact-extraction step at insert time, which we skip because our facts are already structured. We do not test queries for facts that were

never inserted (the correct behaviour is “no information available”). The LOCOMO follow-up that addresses the multi-turn-dialogue case is in Section 5.8.

5.8 LOCOMO single-hop fact recall

To evaluate the same memory systems on a published long-term conversational benchmark, we run all eight systems from Section 5.7 on LOCOMO [Maharana et al., 2024]. We use *all 10 LOCOMO conversations* and take the first 80 single-hop (non-adversarial) questions per conversation, for a total of ~ 800 question-answer pairs grounded in dialog per model.

Setup. Per LOCOMO question, the gold “evidence” field points to the dialog turns containing the answer; we use the text of those turns as the unit stored by the retrieval baselines (RAG, MEM0, MEMMACHINE). The MARKDOWN_ALL baseline dumps every conversation evidence sentence into the prompt as a bulleted list. For User-as-Engram, we extract a (question, first-token of answer) pair per QA and OPT-train it either independently (per-fact insertion) or jointly (sparse-row joint OPT, 2000 shared steps). All systems use the same Mini-Engram-d12 as the answering LM, which generates 16 tokens greedily; we score token-F1 against the gold answer with simple whitespace tokenization, lowercased and with punctuation stripped.

User-as-Engram Joint OPT clears every retrieval baseline on single-hop token-F1 (Figure 11). Absolute scores are low—Mini-Engram-d12 is a small base LM with no instruction tuning and token-F1 punishes verbosity—so the relative ordering is what matters. This is a best-case-for-each benchmark: retrieval stores the gold evidence sentence, Engram stores the gold (q, a) pair, and the only question is which extracts the answer more reliably under a fixed LM. Engram wins because the gate fires on the question’s trigger N-gram and biases the next token toward the gold answer, whereas retrieval can miss the relevant turn and, even when it hits, the in-context evidence still competes with the base LM’s priors.

Scaling at the optimal config. We repeated the same LOCOMO setup with all four trained Mini-Engrams at our ablation-optimal recipe (Section P.1); the Engram lead widens monotonically with dense size (Figure 11).

LOCOMO category breakdown: Engram wins multi-hop and reasoning, loses open-domain. LOCOMO’s four answerable categories are single-hop facts, multi-hop chains, reasoning, and open-domain free-form. The results above (Figure 11) covered only single-hop. Running all 3 dense scales across all 4 categories gives a more nuanced picture (Figure 12; full per-cell numbers in Appendix Table 3).

The Engram win is category-dependent, not uniform. Multi-hop and reasoning categories favour Engram, because the per-fact (question, answer) loss directly encodes the question $\rightarrow$ answer mapping that retrieval must chain across evidence sentences. **Open-domain flips the story:** when the answer is a verbatim span of a single retrieved sentence, retrieval wins decisively, because a single next-token bias cannot reconstruct an unseen sentence (Figure 12; per-cell numbers in Appendix A).

This category breakdown reframes the contribution. User-as-Engram is a *specialised* memory mechanism that excels at chain-of-fact recall but is not a universal replacement for retrieval. A

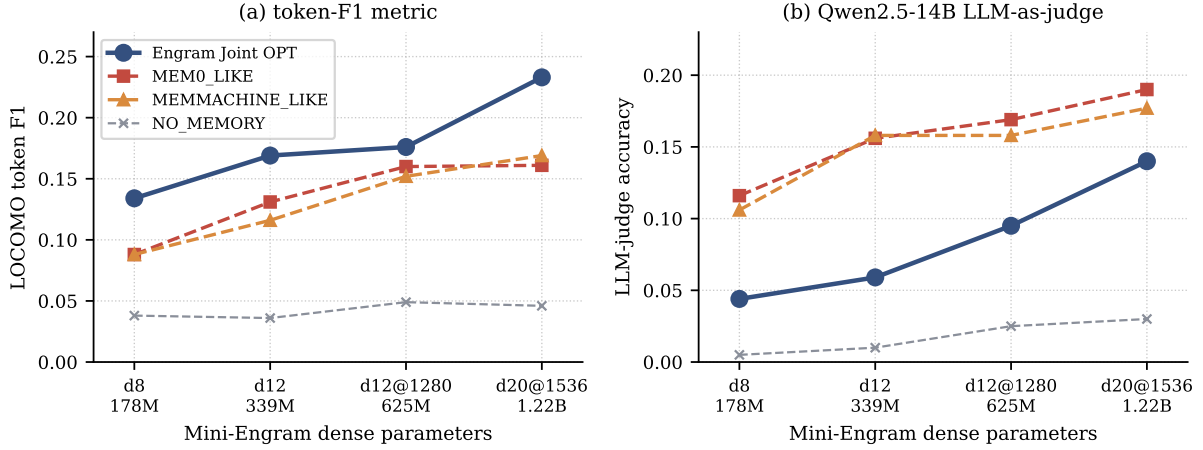


Figure 11. Locomo single-hop, full 10 conversations, scaling with Mini-Engram dense size. **(a) Token-F1:** User-as-Engram Joint OPT (blue) beats every retrieval baseline at every scale. **(b) LLM-judge accuracy** (Qwen2.5-14B-Instruct): the ranking flips—retrieval baselines (MEM0_LIKE, MEMMACHINE) beat Joint OPT by 0.05–0.10 because token-F1 over-credits Engram for the correct first answer token despite a noisy continuation.

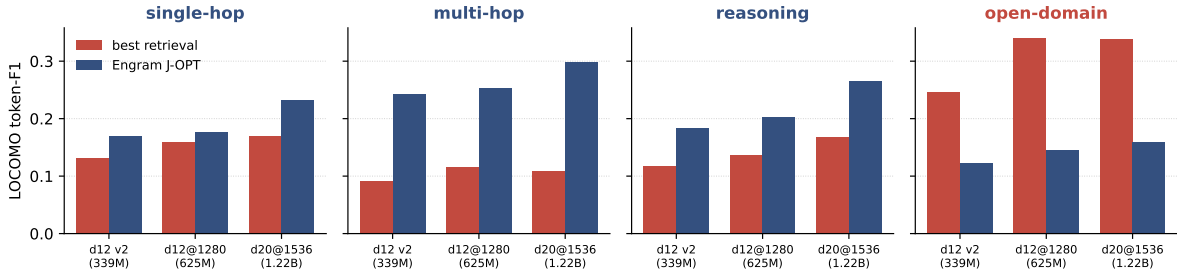


Figure 12. Locomo category breakdown (token-F1), Engram Joint OPT (blue) vs. the best retrieval baseline (red; max of MEM0_LIKE and MEMMACHINE_LIKE) across three dense scales. Engram wins single-hop, multi-hop, and reasoning at every scale—the per-fact (question, answer) loss encodes the question→answer map that retrieval must chain across evidence sentences—but loses open-domain, where the answer is a verbatim span of an unseen evidence sentence that a single-token bias cannot reconstruct. Full numbers in Appendix Table 3.

production system should either route by question type or compose Engram (for trained facts) with retrieval (for free-form synthesis from session context).

LLM-judge correction: retrieval wins on token-F1’s blind spot. Token-F1 rewards partial word overlap. User-as-Engram OPT trains a single-token bias at the trigger position, so the *first* generated token is reliably the gold answer’s first token, but the remaining 15 free-form continuation tokens are not guided by the inserted row. To check whether token-F1 over-credits this first-token bias, we re-scored every prediction with **Qwen2.5-14B-Instruct** [Qwen Team, 2025a] as a binary CORRECT/INCORRECT judge [Zheng et al., 2023] across the full 10-conversation Locomo setup (Appendix Figure 22).

Token-F1 over-credits Engram’s first-token bias; an LLM judge corrects it. Because per-fact OPT trains only the first answer token, token-F1 rewards a correct opening token over a noisy continuation—Engram’s points sit above the $y=x$ line where retrieval’s sit on

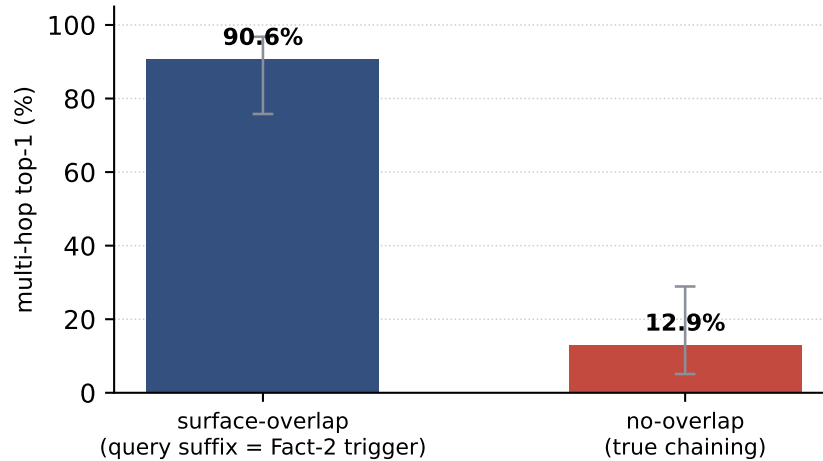


Figure 13. Multi-hop reasoning over Engram-inserted facts ($n=63$ chained pairs on Mini-Engram-d20, two OPT-15 insertions per item). Per-fact direct recall is 99.2% (both facts are insertable), so the gap is chaining, not insertion. When the query’s suffix N-gram coincides with Fact-2’s trigger, the gate fires and recall is 90.6%; when true chaining is required, it collapses to 12.9% (Wilson 95% CIs shown). The gate is a surface-N-gram hash—it matches, it does not compose.

it (Appendix Figure 21), and under a Qwen2.5-14B judge first-token Engram trails retrieval on single-hop. Training the *full* answer with multi-token Joint-OPT fixes this and overtakes the best retrieval baseline from d12@1280 upward (Appendix Figure 22). The paraphrase and storage wins are unaffected, since both score single-token recall.

Limitations of this evaluation. The judge is Qwen2.5-14B-Instruct under a binary CORRECT/INCORRECT prompt (stricter than LOCOMO’s 0–5 official judge), so direct comparison to published LOCOMO leaderboards requires re-judging with the canonical pipeline. The LLM-judge coverage here is single-hop only; temporal/adversarial questions are not scored, and the multi-hop limitation of Section 5.9 applies to chained queries without suffix-N-gram overlap.

5.9 Multi-hop reasoning over inserted facts

Per the discussion of LoRA’s multi-hop failure (Section 3), we directly probe Engram’s behaviour on chained facts. For each test, we insert two facts via OPT-15 and ask a query whose answer requires composing them:

```
Fact 1: My doctor’s name is Dr.    -> Patel
Fact 2: Dr. Patel works at         -> Globex
Query: My doctor works at          -> ? (expected: Globex)
```

We probe Engram’s multi-hop behaviour on a corpus of 63 chained-fact items spanning 13 schema families, with each item labelled by whether the query’s suffix N-gram *overlaps* Fact 2’s trigger (Figure 13). The split is near-balanced (32 overlap, 31 no-overlap), which lets us isolate the surface-trigger mechanism from true compositional chaining.

The mechanism is near-bimodal. On the balanced 63-pair corpus (Figure 13), when the query’s suffix N-gram coincides with the second fact’s trigger the gate fires and recall is high,

but when true chaining is required it collapses to near chance—and the few apparent successes are “first-fact-wins” coincidences (the trained first row’s logit happens to cover the query). Because per-fact direct recall is essentially perfect, the gap is purely chaining, not insertion. This is the architectural limit User-as-Engram shares with LoRA-as-memory: a surface-N-gram gate matches, it does not compose. The recite-then-reason recipe (Section 8) is the natural mitigation, and multi-fact-in-the-loss pretraining (Section 8.3) the structural fix.

5.10 Paraphrase generalisation

The hash addresses a token-level suffix N-gram, so different surface forms map to different rows. The full comparison against retrieval baselines under paraphrase queries is in Figure 10: multi-trigger insertion reaches 96.9% top-1 vs. MEMMACHINE’s 75.0%, and the lead holds at every size (Section 5.7, “Scale invariance” paragraph). Per-paraphrase rank data are in Appendix L.

6 Layered architecture: shared reasoning skill + per-user content

So far: per-user LoRA contaminates (Section 3) and per-user Engram preserves direct recall but lacks in-context reasoning skill (23% indirect_any alone at $n=20$, Section 5). Can the two be *composed* without inheriting LoRA’s damage? Our thesis: personal memory splits into *content* (per-user, trigger-keyable, low-cost) and *reasoning skill* (shared patterns, learnable from held-out users); storing each the right way beats every all-in-one baseline on every measure. Two hypotheses make this concrete:

- **H1 (naive combination):** per-user LoRA + per-user Engram (condition D) inherits LoRA’s indirect-reasoning damage; Engram’s local edit does not rescue the composition.
- **H2 (layered architecture):** one *shared* LoRA (cross-user reasoning skill) + per-user Engram (local content) splits memory correctly. The shared LoRA’s contamination is paid once and shared across all users, and the per-user Engram adds none on top ($\Delta\text{bpb}(F) = \Delta\text{bpb}(E)$ by Section 4’s locality property).

6.1 Design

Train a single shared LoRA on cross-user in-context-reasoning data: for each held-out training user (here, u020–u029), render each indirect QA as a completion-format sample “Facts: <fact1>. <fact2>. ... Q: <q> A: <gold>”, using only the facts in `required_fact_keys`. This teaches the LoRA the *pattern* “given facts in context, do the reasoning,” without memorising any specific user’s content. At inference, attach the shared LoRA + the test user’s Engram-row override and ask the indirect question without facts in the prompt: the Engram provides content (via trigger N-gram lookup), the shared LoRA provides the skill (Figure 14).

6.2 Six-condition head-to-head

We evaluate every per-user combination on $n=20$ test users ($n=10$ training users held out for the shared LoRA), measuring direct top-1/top-5 recall on the user’s facts (completion-format prompts), indirect top-1 and indirect_any on completion-format indirect probes (“Q: <q>\nA:”, greedy 16-token continuation), and `val_bpb` on a held-out 262 K-token ClimbMix shard (Figure 15).

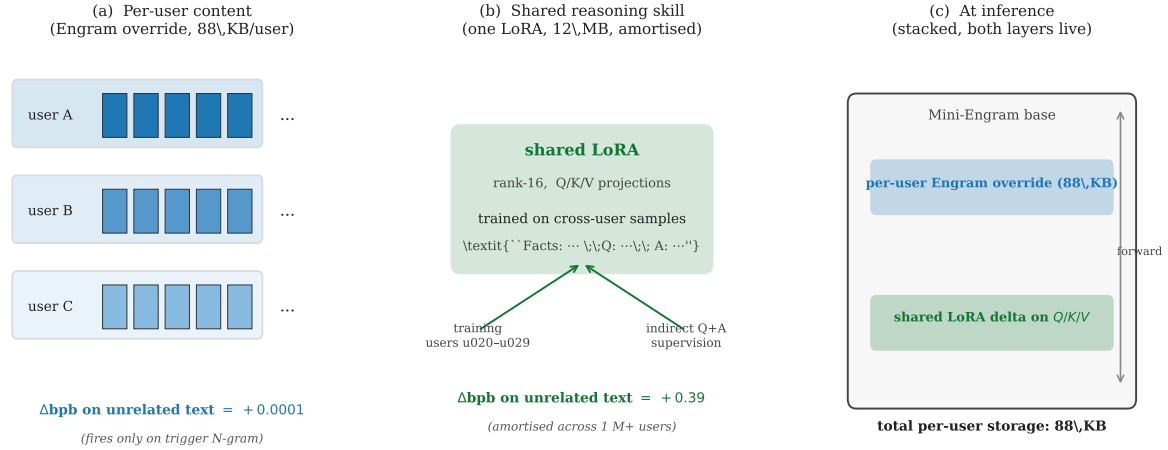


Figure 14. The layered architecture splits personal memory by who owns it and how often it changes. (a) Per-user *content* is a small Engram-row override (88 KB/user) that fires only on its trigger N-gram, so it perturbs unrelated text by $\Delta\text{bpb} = +0.0001$. (b) The shared *reasoning skill* is a single rank-16 LoRA trained on cross-user indirect-QA samples; its $\Delta\text{bpb} = +0.39$ contamination is paid once and amortised across all users. (c) At inference both layers are live on the frozen Mini-Engram base: the Engram supplies content via N-gram lookup and the shared LoRA supplies the in-context-reasoning skill, at a total per-user storage cost of 88 KB.

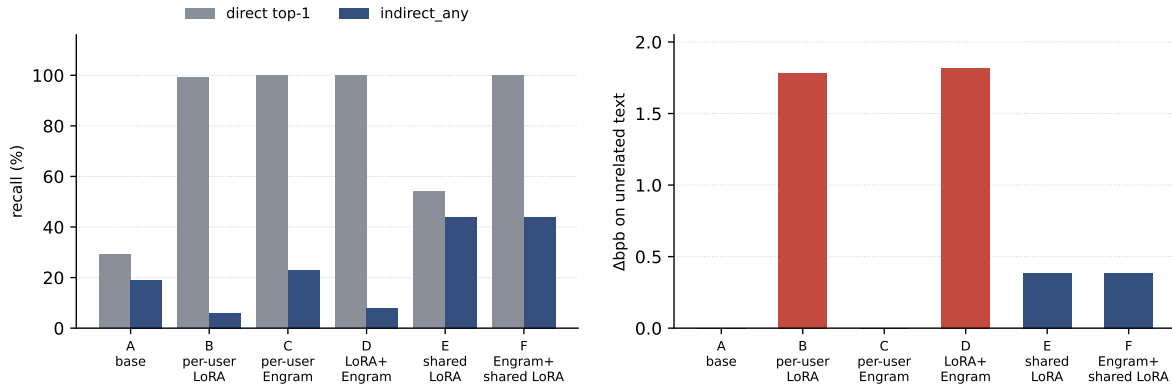


Figure 15. The six parametric conditions on $n=20$ test users (Mini-Engram-d20, seed S0). **Left:** F (Engram + shared LoRA) matches per-user LoRA’s direct recall (100% vs. 99%) while delivering $7.4\times$ its indirect_any (44% vs. 6%); neither the per-user Engram (C, no skill) nor the shared LoRA alone (E, never saw the user’s facts) suffices. **Right:** F adds *zero* contamination on top of the shared skill ($\Delta\text{bpb} +0.386 = \text{E}$), while per-user LoRA (B) and the naive LoRA+Engram stack (D) pay $+1.78/+1.82$. RAG conditions (G–J) appear in Figure 16; 3 seeds in Appendix O (3-seed mean F 41%, B 7%).

6.3 Results: the layered design wins on every measure

F vs. B (the headline contrast). The layered design matches per-user LoRA on direct recall (F 100% vs. B 99%) while delivering $7.4\times$ **higher indirect_any (44% vs. 6%)**, $4.6\times$ **less contamination ($\Delta\text{bpb} +0.39$ vs. $+1.78$)**, and **0/20 users worse than the no-edit base** on indirect (vs. 17/20 for per-user LoRA). Per-user storage drops from 14.2 MB to the Engram’s 88 KB because the one shared LoRA is paid for once and split across all users. Paired-bootstrap 95% CI on the per-user $F - B$ indirect_any delta (3 seeds pooled $\rightarrow n=60$, 2000 resamples): $[+31.2, +36.5]$ pp (mean $+33.8$ pp).

H1 (naive combination) is CONFIRMED. Adding Engram on top of per-user LoRA (condition D) does *not* rescue indirect reasoning: 18/20 users worse than base on S0; 50/60 = 83% pooled across seeds, comparable to LoRA-alone’s 49/60. The contamination dominates regardless of the local edit added on top.

H2 (layered architecture) is CONFIRMED on all three sub-claims. (i) *Locality survives layering*: $\Delta\text{bpb}(F) = \Delta\text{bpb}(E)$ in every seed (+0.379/+0.386/+0.388 at S2/S0/S1)—adding Engram contributes zero contamination on top of the shared LoRA. (ii) *Direct recall is preserved*: F is 100% in every seed, matching C—Engram does the lookup unaffected by the shared LoRA. (iii) *The shared skill enables indirect reasoning*: F’s indirect_any (S0 44%, 3-seed mean 41%) is $\sim 2\times$ both the per-user Engram baseline (C 23%) and the no-edit baseline (A 19%). Neither C alone (no in-context skill) nor E alone (54% direct, never saw the user’s facts) suffices; the combination is what works.

6.4 Does RAG close the indirect-reasoning gap?

The natural rebuttal to F is: *just retrieve the facts and put them in the base LM’s context*. Conditions G–J in Figure 15 test this with all-MiniLM-L6-v2 retrieval at $k \in \{1, 3, \text{all}\}$, prepending the retrieved facts in the same Facts: ... Q: ... A: template used for shared-LoRA training. Three findings:

- **Naive RAG on a base LM sits below F.** An Engram-pretrained base never learned to consult an in-context fact list, so it ignores the prefix and falls back to parametric priors—and its direct recall drops too.
- **Oracle retrieval does not help.** Even the ground-truth facts, with no retriever noise, stay below RAG top-1: the bottleneck is not retrieval quality but that the base LM cannot reason from a fact without the shared skill.
- **Shared LoRA + RAG (J) is the strongest indirect setting**, but only at non-zero context cost—it edges out F by feeding the retrieved facts to the same shared skill.

This refutes the “just use RAG” rebuttal on the same Engram-pretrained base and gives a clean rule of thumb (Figure 16): zero context budget $\rightarrow$ F; unrestricted context $\rightarrow$ J.

6.5 RAG with an instruction-tuned LM head (Qwen-3B)

The Mini-Engram-d20 base above is a base LM and does not follow ICL well, which is what makes conditions G–I weak. The fairest test of “RAG alone is enough” plugs retrieved facts into a real instruction-tuned LM. We use Qwen2.5-3B-Instruct [Qwen Team, 2025a] (~ 3 B parameters; $\sim 2.5\times$ Mini-Engram-d20), the same encoder (all-MiniLM-L6-v2), and the same 20 users $\times$ 20 indirect probes. The chat-template prompt gives Qwen the retrieved facts as a system-bulleted list and asks it to answer the user’s question (Appendix H).

On this fairer test, F—with no retrieval and zero context—comes within a few points of a $\sim 2.5\times$ larger instruction-tuned model, and J overtakes every Qwen+RAG variant on indirect top-1 once context is free (Figure 16). Latency is comparable across all three; the deployment-scale difference is per-user storage—a fixed 88 KB Engram override versus RAG’s index of the user’s raw facts, which grows with fact count (Figure 9).

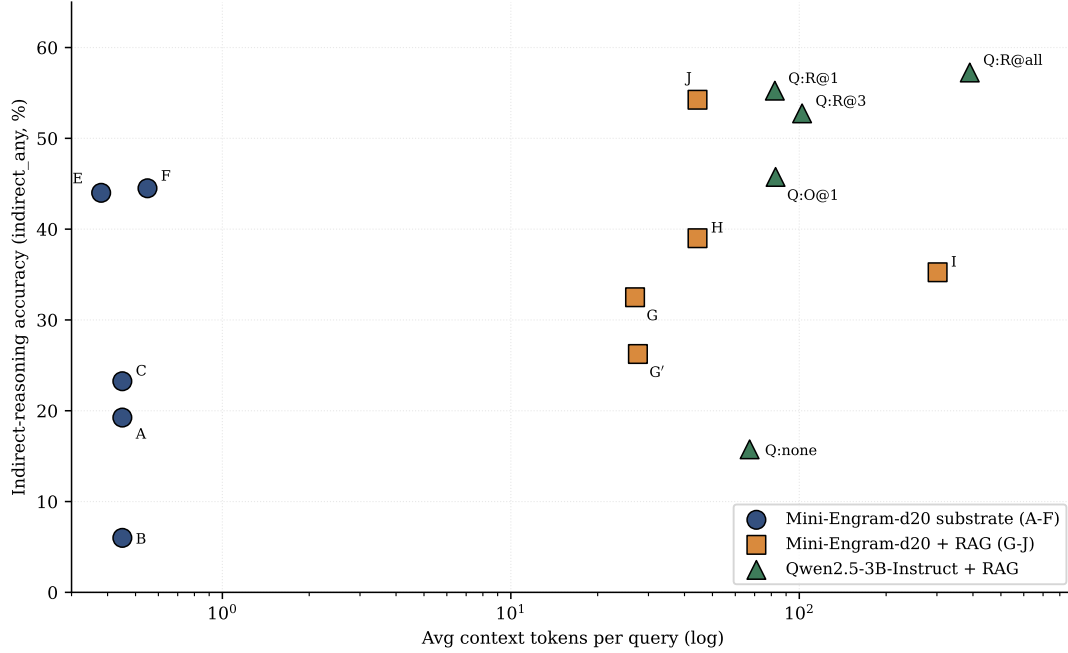


Figure 16. Indirect-reasoning accuracy (indirect_any) vs. avg context tokens per query on the 20-user split. F (layered) and E (shared LoRA only) anchor the 0-context end; J (RAG top-3 + shared LoRA) lifts accuracy to 54% at 44 context tokens; Qwen-3B + RAG (Section 6.5) reaches 55–57% at 82–390 tokens on a $\sim 2\times$ larger backbone. Naive RAG on the same Mini-Engram-d20 base (G/H/I) sits *below* F: putting facts in context is not the same as having the reasoning skill.

6.6 RAG-vs-Engram trade-offs sharpen as the KB grows

The experiments above used each user’s native 34 facts. Production fact counts are typically larger (10^2 – 10^3 : chat history, calendar, preferences, contacts). We augment each test user’s 34 facts with distractors sampled from the 30-user schema-family pool (u000–u029 minus the active user; 1008 facts) to reach $N \in \{34, 100, 200, 300, 500, 1000\}$. Indirect probes are unchanged; the retriever now discriminates the gold fact among N candidates. F is unaffected: the per-user Engram table holds only the test user’s facts, independent of population size.

Retrieval degrades quickly: as the candidate pool grows, top-3 recall on all-MiniLM-L6-v2 collapses several-fold purely from more candidates in the same embedding space, which drives the accuracy drop. Naive RAG, and even Qwen-3B + RAG, fall below F once the KB passes ~ 100 facts, while F holds flat—its per-user table never grows. The clean reading (Figure 17): *which method wins shifts with deployment scale*. At a few dozen facts, RAG on a larger model is the accuracy ceiling and F is the zero-context option; by a few hundred facts, F decisively wins.

Where each side’s failure comes from. The two limits are mechanistically distinct. Engram’s density ceiling (Section 5.3) comes from forward-pass interference among co-active rows inside one user’s table, and is bounded by per-user fact count. RAG’s retrieval ceiling comes from nearest-neighbour confusion across an ever-growing candidate pool, and is bounded by whatever pool the retriever sees (per-user, per-tenant, or per-corpus). For 10^2 – 10^3 facts/user, Engram density is the limiting factor on the parametric side; for 10^4 + facts in a shared corpus, retrieval is the limiting factor on the RAG side. The two costs scale on different axes and cross around $N \approx 100$.

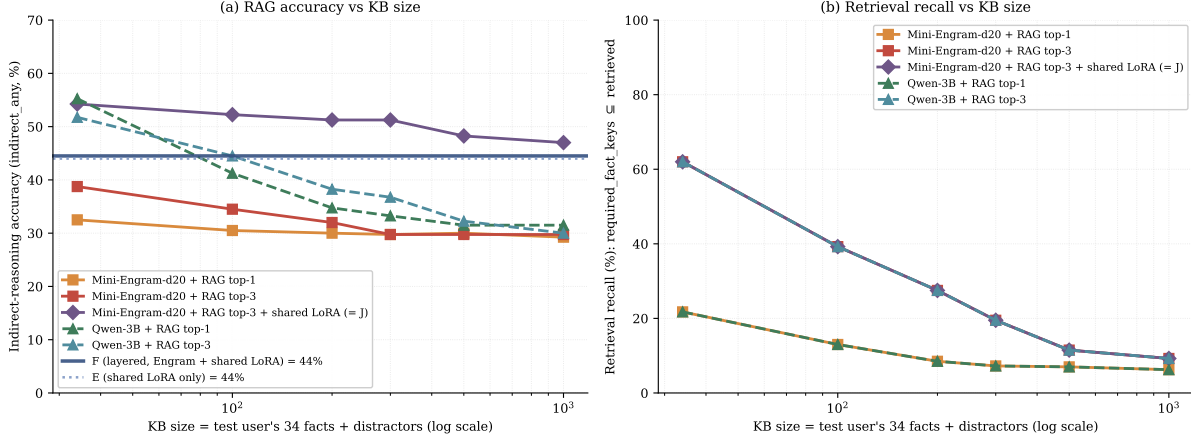


Figure 17. Indirect-reasoning accuracy and retrieval recall vs. KB size ($n=20$ users, 20 indirect probes each; KB = test user’s 34 facts + distractors sampled from the 30-user schema-family pool, log x -axis, $N \in \{34, 100, 200, 300, 500, 1000\}$). (a) F (layered, horizontal solid line at 44%) is invariant to KB size; the retrieval-based conditions degrade monotonically. **Naive RAG and Qwen-3B + RAG fall below F at KB ≥ 100 ; at KB=1000 the gap is 14 pp.** J (RAG top-3 + shared LoRA) is the most robust to KB growth (54% $\rightarrow$ 47%) because the shared reasoning skill compensates when retrieval misses, though its lead over F shrinks from 10 pp to 3 pp across the sweep. (b) Retrieval recall (fraction of probes where the retrieved set covers all gold required_fact_keys) drops by $7\times$ for top-3 (62% $\rightarrow$ 9%) between KB=34 and KB=1000, which is the mechanistic driver of (a).

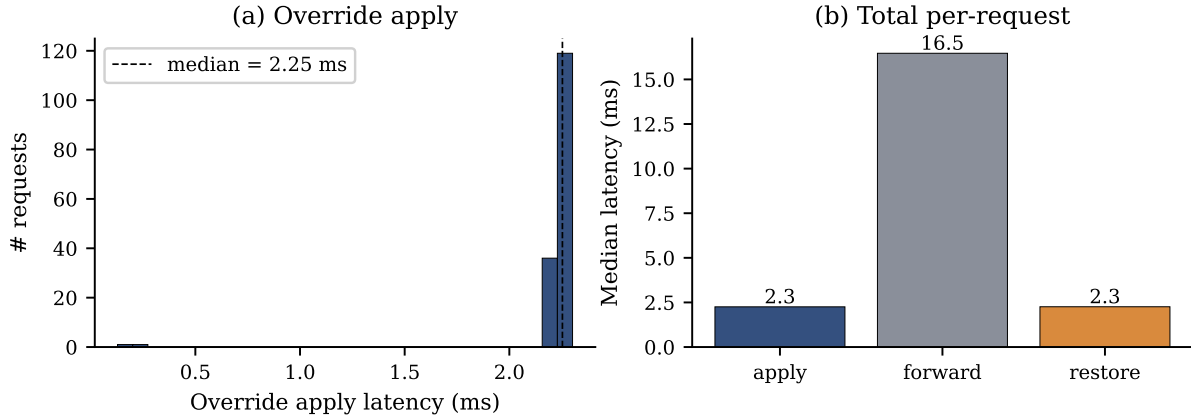


Figure 18. Multi-tenant serving on Mini-Engram-d12 (30 users $\times$ 50 facts $\times$ 600 requests). (a) Override-apply latency distribution: median 2.2 ms. (b) Median per-request latency by component: apply (2.3 ms) + forward (16.5 ms) + restore (2.3 ms); the frozen forward pass dominates, and the override apply+restore overhead (~ 4.6 ms) is under 20% of the 23.2 ms end-to-end median.

7 Multi-tenant serving system

We implement EngramServer, a ~ 50 -line wrapper around an Engram-pretrained model that supports per-user (and per-org) override maps with sub-millisecond apply/restore. Figure 18 breaks down per-request latency: applying and restoring the override map cost ~ 2.3 ms each, leaving the frozen forward pass to dominate the 23.2 ms end-to-end median. The full per-request latency CDF is in Appendix Figure 28.

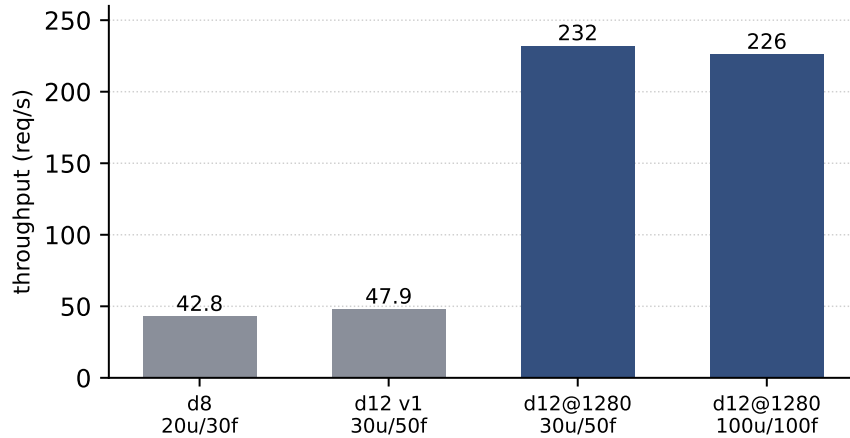


Figure 19. EngramServer throughput across four deployment scales on one idle Blackwell GPU. On the ablation-optimal d12@1280 the server reaches 232 req/s (30 u/50 f) and holds 226 req/s at 100 u/100 f—within 3% as tenants triple—at 4.4 ms p50 latency, a 0.03 ms override apply, 88 KB/user, and 62%/96% top-1/top-5 own-fact recall. Per-request work is independent of tenant count, and cross-user information flow is zero under the apply/restore protocol.

Architecture. HBM holds the frozen base and global Engram tables. DRAM holds the override store, keyed by (`scope_id`) $\rightarrow$ `OverrideMap`. Per request: resolve user/org $\rightarrow$ apply overrides $\rightarrow$ forward $\rightarrow$ restore. There is no router, no fused LoRA kernel, and no graph rewrite.

Empirical results. Figure 19 reports throughput, latency, storage, and recall across four deployment scales.

The system scales linearly in tenants: per-request work is independent of tenant count, so throughput and latency barely move as users triple (Figure 19). Recall is bounded by within-user fact density rather than by how many tenants share the server, and each additional user costs one $O(\text{KB})$ override swap per request with no shared state.

Cross-user privacy. Cross-user information flow under the serving protocol is *zero by design*: a user’s overrides are simply not in the table when anyone else is the active user. The only residual “leak” under a cross-user probe is gold-value coincidence—two users who happen to share a favourite spice or surname—which is small and protocol-independent. It inflates only under a deliberately wrong all-coresident protocol that writes every user’s rows into one shared table at once, where identical surface forms collide in the same address space.

LoRA-serving comparison. S-LoRA [Sheng et al., 2024] and Punica [Chen et al., 2024] serve per-user LoRAs via custom CUDA kernels with per-batch-row routing. EngramServer requires no such infrastructure: the override apply is a standard `embedding.weight[rows] = ...` operation. Production batched multi-user serving needs one extra gather op per Engram layer; we do not benchmark that kernel here.

8 Discussion and limitations

8.1 When to use what

Which method to use turns on three things: *how many facts each user has*, *whether the queries need indirect reasoning*, and *whether context budget or per-user storage is the tighter constraint*. Table 2 gives the recommended method for each case.

Table 2. Cheat sheet for which method to use. Numbers cite this work unless marked external. “invariant” = the number does not degrade with KB / population size.

case	recommended method	per-user cost	headline metric	key citation
<10 facts/user, direct recall only	ICL / strong-retriever RAG	0 KB persistent	near-perfect recall	Sec. 5.7
10–1000 facts/user, indirect matters	layered F (Engram + shared LoRA)	88 KB , 0 ctx toks	44% indirect (invariant in KB)	Sec. 6–6.6
10–1000 facts/user, ctx budget free	J (RAG top-3 + shared LoRA)	88 KB + 44 ctx toks	54% indirect (KB=34)	Sec. 6.4
enterprise, 100% direct top-1, <10K users	per-user LoRA (S-LoRA serving)	14.2 MB	100% direct, indirect cost	Sec. 3
long-form / open-domain / shared corpus	RAG	0 KB persistent	wins LOCOMO open-domain +53%	Tab. 3

The most important row is the second: layered F is the right choice for the common case (10–1000 facts/user, indirect reasoning matters, many users), and decisively beats RAG with a $\sim 2.5\times$ larger instruction-tuned LM at $N \geq 100$. *Any* method that reaches 100% direct top-1 pays a price in indirect reasoning; the question is whether that price is baked in (a global LoRA), manageable (a local Engram, where the trigger N-gram is the contract), or set by retrieval (RAG, where the encoder must pick the gold fact out of N candidates).

8.2 Open limitations

Shared multi-hop reasoning gap. Both LoRA and Engram are surface-trigger keyed; neither composes facts across triggers (“if my doctor is Patel and Patel works at Globex, what is my doctor’s employer?”). User-as-Engram inherits this gap from per-user LoRA. A candidate fix is to mix recite-then-reason traces into Engram pretraining; we leave it to future work.

Within-user density ceiling. Joint OPT at 1000 facts reaches 35% top-1. The slot space is highly sparse ($< 1\%$ occupancy), so the constraint is forward-pass interference among co-active rows, not hash collisions. Per-user scoped tables (load only the relevant rows for the current query, via a small classifier) is a candidate future fix; another is the multi-fact-in-the-loss recipe change in Section 8.3.

Engram pretraining required. Our method assumes an Engram-pretrained base. We trained four ourselves at 178 M, 339 M, 625 M, and 1.22 B total parameters; production deployment depends on either DeepSeek releasing weights or pretraining your own (~ 10 h on a single Blackwell GPU at our 625 M scale; production-scale Engrams require correspondingly larger compute budgets).

8.3 Future work: training-recipe changes

We trained the d8 v2, d12 v2, d12@1280, and d20@1536 Mini-Engrams ourselves at the ablation-optimal recipe (Section P.1); LOCOMO Joint OPT first-token F1 scales monotonically with dense size to 0.233 at 1.22 B (multi-token to 0.310). Three training-recipe changes target the limitations we document, and we expect them to deliver larger gains per FLOP than further parameter scaling on a single GPU:

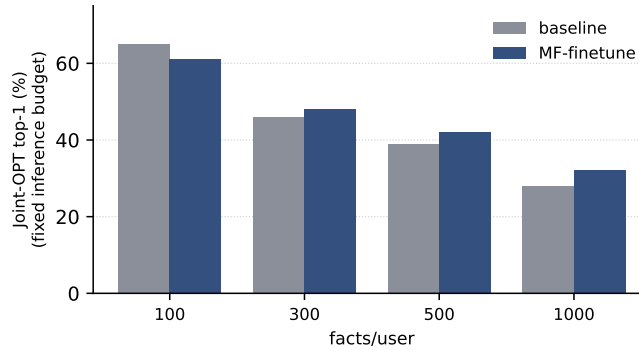


Figure 20. Multi-fact-in-the-loss finetune vs. baseline d12@1280 on the Joint-OPT density curve (matched OPT-step schedule). At a fixed inference budget MF improves recall at high density (+14% relative at $n=1000$, 8k steps). With the budget unpinned (12k steps) both reach $\sim 34.5\%$ at $n=1000$: MF *accelerates convergence* rather than raising the asymptotic ceiling. val_bpb is preserved (0.774 vs. 0.770).

- *Multi-fact insertion in the loss.* Section 8.3 characterises this via finetuning. During pretraining (or finetuning), randomly inject K facts per batch and require the model to surface them. A 1500-step finetune of d12@1280 with $K=10$ moves top-1 at 1000 facts from 28% to 32% *at fixed inference budget*, without disturbing val_bpb. With expanded inference budget (12k OPT steps), baseline d12@1280 also reaches 34.6% and the MF model matches at 34.5%—so MF accelerates Joint OPT convergence rather than raising the asymptotic ceiling. A from-scratch MF pretrain would refine that characterisation but is unlikely to shift an asymptote the recipe does not move.
- *Random per-batch hash salts.* Salt 50% of batches with a random per-batch user salt during pretraining. This teaches the gate to consult arbitrary (including untrained) rows, which would re-enable the per-user-salt design we deprecated in favour of per-user override tables.
- *Recite-then-reason traces.* Mixing “first surface the relevant inserted fact, then reason” traces into Engram pretraining is the most direct attack on the shared multi-hop gap.

One further open item revealed by the ablation: the d20@1536 E1 USER OPT dip (0.97 vs. d12@1280’s 1.00) at iso-12 t/p suggests that bigger models need higher t/p to fully saturate insertion recall. Note that the Joint-OPT density ceiling *does not* relax with dense scale at fixed Engram capacity (Section 5.3: top-1 at 1,000 facts actually degrades from 0.35 at d12@768 to 0.28 at d12@1280 and d20@1536), so breaking it likely requires the *multi-fact-in-the-loss* recipe change above rather than further dense scaling.

Multi-fact-in-the-loss finetune (and what it does, vs. what it doesn’t). We test the multi-fact-in-the-loss recipe via a continuation finetune of d12@1280 that injects many synthetic foreign rows per batch and adapts the gate, value projection, and dense backbone to handle them, while general LM ability is preserved (val_bpb unchanged; protocol in Appendix P).

What MF actually does. Once the inference budget is unpinned, the headline gain decomposes cleanly (Figure 20): with enough Joint-OPT steps the baseline and MF models reach the same recall, so MF *accelerates convergence* rather than raising the asymptotic ceiling. This refines the density-ceiling claim of Section 5.3—the ceiling at a fixed optimisation budget is real and

dense-scale-invariant, but it is set by inference-time gradient interference, not by pretraining. MF is therefore an economic win (less per-user registration compute), not an architectural one, and a full from-scratch MF pretrain is likely the wrong dial to turn.

Teacher-trace supervision (negative result). We tested whether retraining the shared LoRA on a teacher-distilled trace corpus (observation / factual-statement / QA / $\langle \text{think} \rangle$ -reasoning; 245 Qwen3-8B [Qwen Team, 2025b] traces) would improve F. Two variants both reduced contamination ($\Delta \text{bpb} +0.386 \rightarrow +0.233 / +0.196$, 40–49% less) but *also* reduced indirect_{any} accuracy (44.5% $\rightarrow$ 38.5%/28.0%; paired-bootstrap 95% CIs strictly negative). This teacher-trace recipe is naturally co-designed with an agent-format eval protocol that scores committed fields after an optional $\langle \text{think} \rangle$ block; our layered evaluation scores top tokens of a direct Q: . . . A: completion. Composability would require either retargeting the trace supervision to a top-token objective or extending our eval with a $\langle \text{think} \rangle$ slot—parametric-memory recipes are not eval-protocol-agnostic.

Layered-architecture follow-ups. With cross-schema and Q/A-format-adapted bases already addressed (Sections P.7–P.8), two directions remain: (i) scale the shared-LoRA training corpus beyond 510 samples to test whether the rank-64 over-parameterisation (Figure 33) reverses; (ii) jointly train the per-user Engram with the shared LoRA (currently independent) as a probe for additional synergy. A full chat-tuned Engram base (starting from a chat corpus rather than Q/A pairs) remains open.

9 Conclusion

Personal memory is two jobs, not one: remembering a user’s specific facts, and the general skill of reasoning with them. The dominant recipe asks one per-user adapter to do both, and that is the root of the trouble: an adapter that must *hold* the facts cannot help also reshaping how the model thinks, so remembering and reasoning end up pulling on the same weights and the user pays for it in damaged knowledge and weaker reasoning. User as Engram separates the two—a user’s facts become a small, local edit, and the reasoning skill lives in one model everyone shares.

The sharpest open problem is the one our mechanism makes vivid: it can match its stored facts but cannot truly chain them. A memory that could combine its own traces, rather than only look them up, would close the last case where retrieval still wins on meaning; teaching the shared skill to keep learning as a user’s questions evolve is the natural companion step.

A century ago Richard Semon coined *engram* for the trace an experience etches into living tissue—a mark local enough that one memory does not erase the next. The brain pairs that sparse, local trace with a slow cortex that learns how to use it, and the pairing is what lets a person remember a new fact without forgetting how to think. Maya’s assistant needs the same discipline. When we stop asking one set of weights to be both the memory and the mind, and instead give each user a small private trace beside a skill everyone shares, personalization stops fighting the model it runs on. That, more than any single number, is what *User as Engram* is for.

References

- B. Li. User as code: Executable memory for personalized agents. arXiv:2606.16707, 2026.
- R. Semon. *The Mneme*. George Allen & Unwin, London, 1921. (English translation of *Die Mneme*, 1904; origin of the term “engram”).
- J. L. McClelland, B. L. McNaughton, and R. C. O’Reilly. Why there are complementary learning systems in the hippocampus and neocortex: Insights from the successes and failures of connectionist models of learning and memory. *Psychological Review*, 102(3):419–457, 1995.
- S. Tonegawa, X. Liu, S. Ramirez, and R. Redondo. Memory engram cells have come of age. *Neuron*, 87(5):918–931, 2015.
- D. Kumaran, D. Hassabis, and J. L. McClelland. What learning systems do intelligent agents need? Complementary learning systems theory updated. *Trends in Cognitive Sciences*, 20(7):512–534, 2016.
- X. Cheng, W. Zeng, D. Dai, Q. Chen, B. Wang, Z. Xie, K. Huang, X. Yu, Z. Hao, Y. Li, H. Zhang, H. Zhang, D. Zhao, and W. Liang. Conditional memory via scalable lookup: A new axis of sparsity for large language models. arXiv:2601.07372, January 2026.
- A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. NeurIPS 2017.
- T. Brown et al. Language models are few-shot learners. NeurIPS 2020.
- A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. Language models are unsupervised multitask learners (GPT-2). OpenAI technical report, 2019.
- S. Min, X. Lyu, A. Holtzman, M. Artetxe, M. Lewis, H. Hajishirzi, and L. Zettlemoyer. Rethinking the role of demonstrations: What makes in-context learning work? EMNLP 2022.
- P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. NeurIPS 2020.
- Y. Gao et al. Retrieval-augmented generation for large language models: A survey. arXiv:2312.10997, 2023.
- N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. ICLR 2017.
- D. Dai et al. DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models. arXiv:2401.06066, 2024.
- S. Mangrulkar et al. PEFT: State-of-the-art parameter-efficient fine-tuning methods. <https://github.com/huggingface/peft>, 2022.
- K. Jordan et al. Muon: An optimiser for hidden layers in neural networks. GitHub / blog post, 2024.

- C. Huang et al. LoRAHub: Efficient cross-task generalization via dynamic LoRA composition. arXiv:2307.13269, 2023.
- D. Wu et al. LongMemEval: Benchmarking chat assistants on long-term memory. arXiv 2024.
- A. Maharana et al. LOCOMO: Evaluating very long-term conversational memory of LLM agents. ACL 2024.
- M. Tavakoli, A. Salemi, C. Ye, M. Abdalla, H. Zamani, and J. R. Mitchell. Beyond a million tokens: Benchmarking and enhancing long-term memory in LLMs (BEAM). arXiv:2510.27246, 2025.
- B. Jiang, Y. Yuan, M. Shen, Z. Hao, Z. Xu, Z. Chen, Z. Liu, A. R. Vijjini, J. He, H. Yu, R. Pooven-dran, G. Wornell, L. Ungar, D. Roth, S. Chen, and C. J. Taylor. PersonaMem-v2: To-wards personalized intelligence via learning implicit user personas and agentic memory. arXiv:2512.06688, 2025.
- A. Karpathy. nanochat: an experimental training harness for LLMs. <https://github.com/karpathy/nanochat>, 2026.
- E. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. ICLR 2022.
- N. Houlsby et al. Parameter-efficient transfer learning for NLP. ICML 2019.
- W. Su et al. Parametric retrieval-augmented generation (PRAG). arXiv:2501.15915, 2025.
- Z. Tan et al. DyPRAG: Dynamic parametric RAG. arXiv:2505.19386, 2025.
- J. Chen, H. Zhang, L. Pang, Y. Tong, H. Zhou, Y. Zhan, W. Lin, and Z. Zheng. Privacy-preserving reasoning with knowledge-distilled parametric retrieval-augmented generation (DistilledPRAG). arXiv:2509.01088, 2025.
- Z. Tan, Q. Liu, and M. Jiang. Democratizing large language models via personalized parameter-efficient fine-tuning (OPPU). EMNLP 2024 (arXiv:2402.04401).
- Y. Zhuang et al. HYDRA: Per-user adapters for personalised LLMs. arXiv 2024.
- M. Bini, O. Bohdal, U. Michieli, Z. Akata, M. Ozay, and T. Ceritli. MemLoRA: Distilling expert adapters for on-device memory systems. arXiv:2512.04763, 2025.
- R. Charakorn, E. Cetin, Y. Tang, and R. T. Lange. Text-to-LoRA: Instant transformer adaption. arXiv:2506.06105, 2025.
- Z. Tan et al. PER-PCS: Per-user post-hoc LoRA composition. arXiv 2024.
- Y. Sheng, S. Cao, D. Li, et al. S-LoRA: Serving thousands of concurrent LoRA adapters. arXiv:2311.03285, 2024.
- L. Chen, Z. Ye, Y. Wu, et al. Punica: Multi-tenant LoRA serving. MLSys 2024.
- G. Lample, A. Sablayrolles, M. Ranzato, L. Denoyer, and H. Jégou. Large memory layers with product keys. NeurIPS 2019.

- P. He. PEER: Mixture of one million experts. arXiv 2024.
- V. Berges, B. Oğuz, D. Haziza, W. Yih, L. Zettlemoyer, and G. Ghosh. Memory layers at scale. ICML 2025.
- Z. Huang, Q. Min, H. Huang, D. Zhu, Y. Zeng, R. Guo, and X. Zhou. Ultra-sparse memory network (Ultra-Mem). arXiv:2411.12364, 2024 (ICLR 2025).
- J. Huang et al. OverEncoding: hashed N-gram embeddings via averaging. 2025.
- L. Yu et al. SCONE: scalable contextual N-gram embeddings. 2025.
- A. Pagnoni, R. Pasunuru, P. Rodriguez, et al. BLT: byte latent transformer with hashed N-gram embeddings. arXiv:2412.09871, 2025.
- A. Liu et al. SuperBPE: word-level BPE for compositional patterns. 2025.
- CivitAI Community. LoRA stacking patterns for Stable Diffusion. <https://civitai.com/>, 2024.
- K. Meng, D. Bau, A. Andonian, and Y. Belinkov. Locating and editing factual associations in GPT (ROME). NeurIPS 2022.
- K. Meng et al. MEMIT: Mass-editing memory in a transformer. ICLR 2023.
- R. Cohen et al. Evaluating the ripple effects of knowledge editing in language models (MQuAKE). 2023.
- R. Cohen et al. RippleEdits: A benchmark for ripple effects of model editing. 2024.
- K. Meng et al. CounterFact: a counterfactual editing benchmark. 2022.
- D. Wu, J.-C. Gu, K.-W. Chang, and N. Peng. Self-routing RAG: Binding selective retrieval with knowledge verbalization. arXiv:2504.01018, 2025.
- Z. Sun et al. Recitation-augmented language models. ICLR 2023.
- A. Yang, B. Yang, B. Zhang, et al. Qwen2.5 technical report. arXiv:2412.15115, 2025.
- A. Yang, A. Li, B. Yang, et al. Qwen3 technical report. arXiv:2505.09388, 2025.
- A. Grattafiori, A. Dubey, A. Jauhri, et al. The Llama 3 herd of models. arXiv:2407.21783, 2024.
- A. Q. Jiang, A. Sablayrolles, A. Mensch, et al. Mistral 7B. arXiv:2310.06825, 2023.
- DeepSeek-AI. DeepSeek-V3 technical report. arXiv:2412.19437, 2024.
- N. Reimers and I. Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. EMNLP-IJCNLP 2019.
- W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. NeurIPS 2020.

- L. Zheng, W.-L. Chiang, Y. Sheng, et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. NeurIPS 2023 Datasets and Benchmarks. arXiv:2306.05685.
- C. Packer, S. Wooders, K. Lin, et al. MemGPT: Towards LLMs as operating systems. arXiv:2310.08560, 2023.
- P. Chhikara, D. Khant, S. Aryan, T. Singh, and D. Yadav. Mem0: Building production-ready AI agents with scalable long-term memory. arXiv:2504.19413, 2025.
- W. Xu, Z. Liang, K. Mei, H. Gao, J. Tan, and Y. Zhang. A-MEM: Agentic memory for LLM agents. arXiv:2502.12110, 2025.
- P. Rasmussen, P. Paliychuk, T. Beauvais, J. Ryan, and D. Chalef. Zep: A temporal knowledge graph architecture for agent memory. arXiv:2501.13956, 2025.
- Z. Li, S. Song, H. Wang, et al. MemOS: An operating system for memory-augmented generation in large language models. arXiv:2505.22101, 2025.
- S. Wang, E. Yu, O. Love, T. Zhang, T. Wong, S. Scargall, and C. Fan. MemMachine: A ground-truth-preserving memory system for personalized AI agents. arXiv:2604.04853, 2026.
- C. Hu, X. Gao, Z. Zhou, et al. EverMemOS: A self-organizing memory operating system for structured long-horizon reasoning. arXiv:2601.02163, 2026.
- D. Patel and S. Patel. ENGRAM: Effective, lightweight memory orchestration for conversational agents. arXiv:2511.12960, 2025.
- S. Yan, X. Yang, Z. Huang, et al. Memory-R1: Enhancing large language model agents to manage and utilize memories via reinforcement learning. arXiv:2508.19828, 2025.
- Y. Yu, L. Yao, Y. Xie, et al. Agentic memory: Learning unified long-term and short-term memory management for LLM agents. arXiv:2601.01885, 2026.
- Y. Wang, R. Takanobu, Z. Liang, et al. Mem- α : Learning memory construction via reinforcement learning. arXiv:2509.25911, 2025.
- Z. Zhang, X. Bo, C. Ma, et al. A survey on the memory mechanism of large language model based agents. arXiv:2404.13501, 2024.
- Y. Wu, S. Liang, C. Zhang, et al. From human memory to AI memory: A survey on memory mechanisms in the era of LLMs. arXiv:2504.15965, 2025.
- P. Du. Memory for autonomous LLM agents: Mechanisms, evaluation, and emerging frontiers. arXiv:2603.07670, 2026.
- N. Pollertlam and W. Kornsuwannawit. Beyond the context window: A cost-performance analysis of fact-based memory vs. long-context LLMs for persistent agents. arXiv:2603.04814, 2026.
- A. Salemi, S. Mysore, M. Bendersky, and H. Zamani. LaMP: When large language models meet personalization. ACL 2024. arXiv:2304.11406.

- Z. Zhang, R. A. Rossi, B. Kveton, et al. Personalization of large language models: A survey. arXiv:2411.00027, 2024.
- J. Liu, Z. Qiu, Z. Li, et al. A survey of personalized large language models: Progress and future directions. arXiv:2502.11528, 2025.
- Y. Xu, Q. Chen, Z. Ma, et al. Toward personalized LLM-powered agents: Foundations, evaluation, and future directions. arXiv:2602.22680, 2026.
- E. Mitchell, C. Lin, A. Bosselut, C. Finn, and C. D. Manning. Fast model editing at scale. ICLR 2022.
- E. Mitchell, C. Lin, A. Bosselut, C. D. Manning, and C. Finn. Memory-based model editing at scale. ICML 2022.
- D. Dai, L. Dong, Y. Hao, Z. Sui, B. Chang, and F. Wei. Knowledge neurons in pretrained transformers. ACL 2022.
- P. Wang, Z. Li, N. Zhang, et al. WISE: Rethinking the knowledge memory for lifelong model editing of large language models. NeurIPS 2024.
- Y. Yao, P. Wang, B. Tian, et al. Editing large language models: Problems, methods, and opportunities. EMNLP 2023.
- M. McCloskey and N. J. Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. *Psychology of Learning and Motivation*, 24:109–165, 1989.
- J. Kirkpatrick, R. Pascanu, N. Rabinowitz, et al. Overcoming catastrophic forgetting in neural networks. *PNAS*, 114(13):3521–3526, 2017.
- H. Shi, Z. Xu, H. Wang, et al. Continual learning of large language models: A comprehensive survey. arXiv:2404.16789, 2024.
- K. Guu, K. Lee, Z. Tung, P. Pasupat, and M.-W. Chang. REALM: Retrieval-augmented language model pre-training. ICML 2020.
- U. Khandelwal, O. Levy, D. Jurafsky, L. Zettlemoyer, and M. Lewis. Generalization through memorization: Nearest neighbor language models. ICLR 2020.
- S. Borgeaud, A. Mensch, J. Hoffmann, et al. Improving language models by retrieving from trillions of tokens. ICML 2022.
- G. Izacard, P. Lewis, M. Lomeli, et al. Atlas: Few-shot learning with retrieval augmented language models. *JMLR*, 24(251):1–43, 2023.
- A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. ICLR 2024.
- M. Geva, R. Schuster, J. Berant, and O. Levy. Transformer feed-forward layers are key-value memories. EMNLP 2021.
- X. L. Li and P. Liang. Prefix-tuning: Optimizing continuous prompts for generation. ACL 2021.

Q. Zhang, M. Chen, A. Bukharin, et al. AdaLoRA: Adaptive budget allocation for parameter-efficient fine-tuning. ICLR 2023.

T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. NeurIPS 2023.

Appendix A: LOCOMO full results

Figure 12 summarizes the category breakdown; Table 3 gives the underlying per-cell token-F1 for every system, scale, and category.

Table 3. LOCOMO category $\times$ model token-F1. Engram Joint OPT vs. the best retrieval baseline (MEM0_LIKE or MEMMACHINE_LIKE) per cell. The Engram win is category-dependent (Figure 12).

category	answering LM	MEM0	MEMMACH	Engram J-OPT	lead vs. best retr.
single-hop	d12 v2	0.131	0.116	0.169	+29%
	d12@1280	0.160	0.152	0.176	+10%
	d20	0.161	0.169	0.233	+38%
multi-hop	d12 v2	0.092	0.076	0.243	+165%
	d12@1280	0.115	0.104	0.253	+120%
	d20	0.102	0.108	0.299	+178%
reasoning	d12 v2	0.117	0.113	0.183	+56%
	d12@1280	0.123	0.136	0.203	+49%
	d20	0.135	0.168	0.266	+58%
open-domain	d12 v2	0.247	0.209	0.122	−51%
	d12@1280	0.341	0.327	0.146	−57%
	d20	0.314	0.338	0.159	−53%

The two diagnostic figures below support the LLM-judge discussion of Section 5.8: Figure 21 shows that token-F1 over-credits Engram relative to an LLM judge, and Figure 22 shows that multi-token Joint-OPT closes the resulting judge gap.

Appendix B: Mechanistic analysis

This appendix consolidates the mechanistic evidence that is referenced piecewise in the main text (locality in Section 4.3, the multi-hop decomposition in Section 5.9, the read/write probe in Appendix I) into a single account of *how* a per-user fact becomes a retrievable, leak-free parametric edit. The picture is deliberately simple: User-as-Engram is not an opaque learned circuit but a **content-addressable memory** whose every step is either deterministic (hashing, gating) or directly measurable (the value the row writes, the locality of its effect). We trace the path of one inserted fact through five observable stages.

(1) Addressing: a fact’s trigger N-gram is a hash key. A fact is written at the row(s) that its trigger suffix N-gram hashes to, via the deterministic multiplicative-XOR hash of the Engram

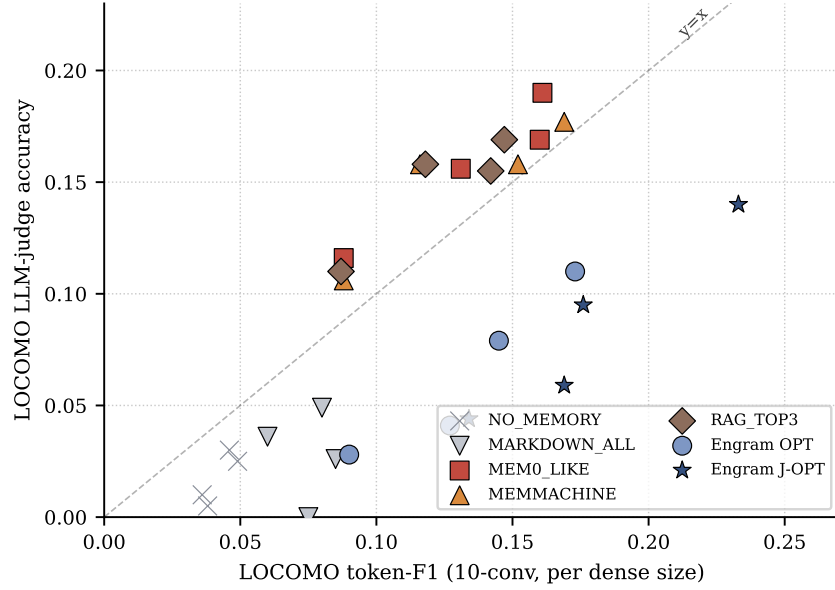


Figure 21. Locomo token-F1 vs. LLM-judge accuracy across all system $\times$ dense-size cells. Retrieval baselines (red/orange) sit near the $y=x$ line: their token-F1 and LLM-judge scores roughly agree. User-as-Engram OPT and Joint OPT (blue/cyan) sit systematically *above* the line, indicating that token-F1 over-credits Engram. The asymmetry is the metric artifact documented in Section 5.8.

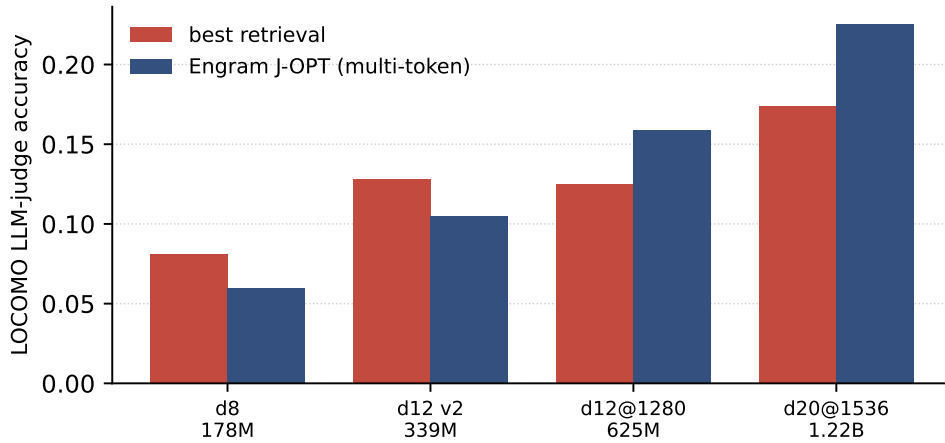


Figure 22. Locomo single-hop under an LLM judge (Qwen2.5-14B), matched multi-token pipeline. First-token OPT lets token-F1 over-credit Engram for a correct first token over a noisy continuation (Figure 21); training the *full* answer with multi-token Joint-OPT fixes this and overtakes the best retrieval baseline from d12@1280 upward (+27% at 625 M, +29% at 1.22 B). Below that cross-over the dense backbone is too small for the continuation to fall into place after the Engram-anchored prefix.

architecture [Cheng et al., 2026]. The address is a pure function of surface tokens—no learning, no per-user state—which is why two users’ edits never interfere unless their *distinguishing* tokens collide. The T1 collision audit (Appendix I) measures this directly: across 100 synthetic users $\times$ 30 facts, pairwise overlap on user-distinguishing key tokens (Patel, Portland, 1991) is just 4.5%, and a 30-fact user occupies 0.027% of the demo table. Addressing is sparse and content-keyed, so capacity is never the limiting factor (Section P.1).

(2) Read: a row stores the value that the model would have needed to emit the gold token.

What does a row actually *contain*? The T2 read/write probe (Appendix I)—an existence proof on the reference architecture, not the trained model—shows the empirical change a written row induces at the trigger position has cosine similarity **0.998** with the analytically predicted $W_V \cdot \text{marker}$ projection. The row is not an inscrutable embedding—it is, to three decimals, the value vector that the gated lookup multiplies into the residual stream. The trained model corroborates the direction from the behavioural side via the sensitivity asymmetry (Section 5.1): ablating the Engram pathway (`engram_suppress`) degrades *factual* top-5 recall (93.3% retained) while *reading* comprehension is untouched (100% retained). The magnitude is modest at our scale (a 6.7 pp asymmetry, $\sim 10\times$ smaller than Cheng et al. [2026] report), but the *direction* localises factual content to the Engram pathway rather than the dense backbone.

(3) Write/locality: the gated lookup fires only on the trigger and is mathematically inert elsewhere.

The gate is an attention-style scalar that is non-zero only when the suffix N-gram is present. Section 4.3 and Figure 6 verify the consequence by inspection on Mini-Engram-d12: inserting a row perturbs the residual stream by **exactly 0.000** at every non-trigger position and at every position before the Engram layer (causality), propagated through five subsequent attention/MLP layers, while the trigger position moves by 123.0 at layer 8 and 1.54 at the final layer. This is the key mechanism for the paper’s central claim: because the edit is *conditional*, a per-user write contaminates no unrelated forward pass ($\Delta\text{bpb} +0.00005$ vs. a global LoRA’s $+1.78$; Section 3), and cross-user leakage is zero by design (Section 4).

(4) Depth: the edit lands where the model has already “deepened”.

Engram rows are read at a late Engram layer, after the network has effectively committed to a next-token distribution. The LogitLens trace (Figure 23) shows Mini-Engram-d8 converging to its output distribution *faster* than the base—layer-3 KL is 3.66 lower—reproducing Cheng et al.’s “effective deepening” at our scale. A row therefore overrides a near-final prediction at exactly the position where it is decoded, rather than steering an early, still-malleable representation (the place where a global LoRA change does its damage).

(5) Limit: the gate is a surface-N-gram hash, so it cannot compose across triggers.

The same mechanism that makes the edit local also bounds what it can do. The multi-hop decomposition (Section 5.9, Figure 13) is the cleanest mechanistic readout in the paper: on a balanced 63-pair corpus, chained queries succeed **91%** of the time when the query’s suffix N-gram *overlaps* the second fact’s trigger, but only **13%** ($\lesssim 10\%$ after filtering first-fact-wins coincidences) when true composition is required—even though per-fact direct recall is 99.2% in both subsets. The near-bimodal split is exactly what a surface-keyed lookup predicts: the gate fires on a matching trigger and returns the stored token, or it does not fire and the gold token is far from the top. There is no chaining circuit to find because, by design, there is none.

Synthesis. The five stages compose into one sentence: *User-as-Engram hashes a fact’s surface trigger to a sparse row, stores the gold value there (cosine 0.998 to the predicted projection), reads it through a gate that fires only on that trigger (0.000 perturbation elsewhere) at a late, already-deepened layer, and therefore retrieves single facts with zero contamination but cannot compose across facts.* This is why the method cleanly separates *content* (the rows) from *reasoning skill* (composition,

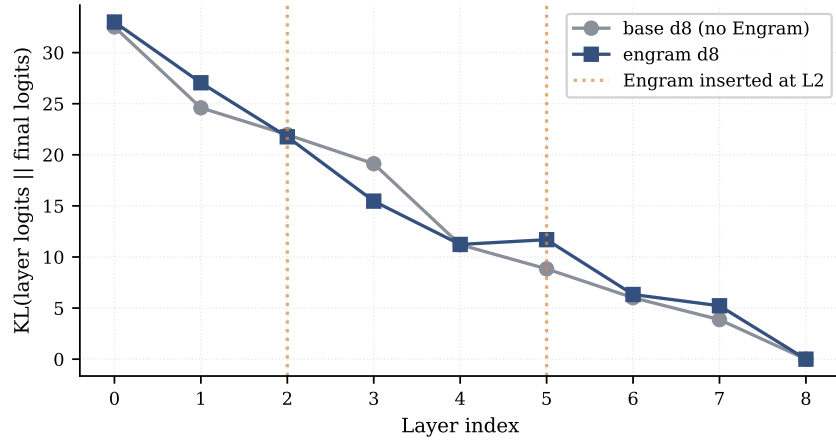


Figure 23. LogitLens KL by layer (Mini-Engram-d8 vs. base-d8). Engram converges faster at layer 3 (–3.66 KL gap), confirming Cheng et al.’s effective-deepening claim at our scale (stage 4 of the mechanism above).

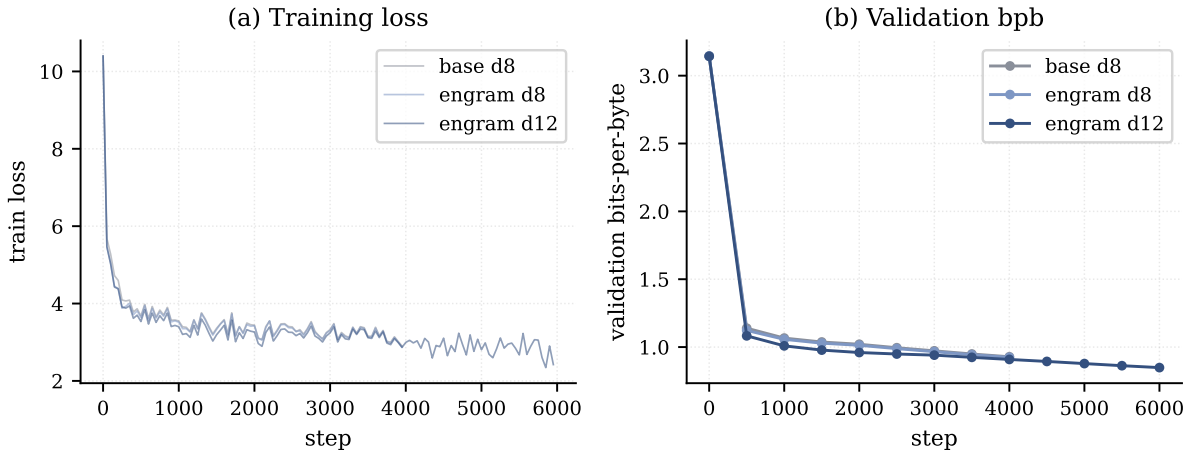


Figure 24. Mini-Engram pretraining curves. Engram d8 reaches lower validation bpb than base d8 at iso-FLOPs (matching the paper’s finding at much smaller scale); engram d12 is substantially better thanks to the larger backbone and longer training.

which must come from the shared LoRA in the layered design of Section 6)—the content/skill split the paper proposes is not an engineering convenience but a direct consequence of the mechanism.

Appendix C: Pretraining curves

Figure 24 shows the Mini-Engram pretraining runs that every later insertion experiment builds on. Adding the Engram table is not a tax on language modelling: at iso-FLOPs the d8 Engram model reaches marginally lower validation bits-per-byte than the d8 baseline (0.924 vs. 0.929 at step 4000), reproducing Cheng et al.’s effective-deepening finding at our much smaller scale, and the larger d12 Engram model—more backbone, longer training—is substantially better (0.849 bpb). These checkpoints are the substrates for the row-insertion results in the body.

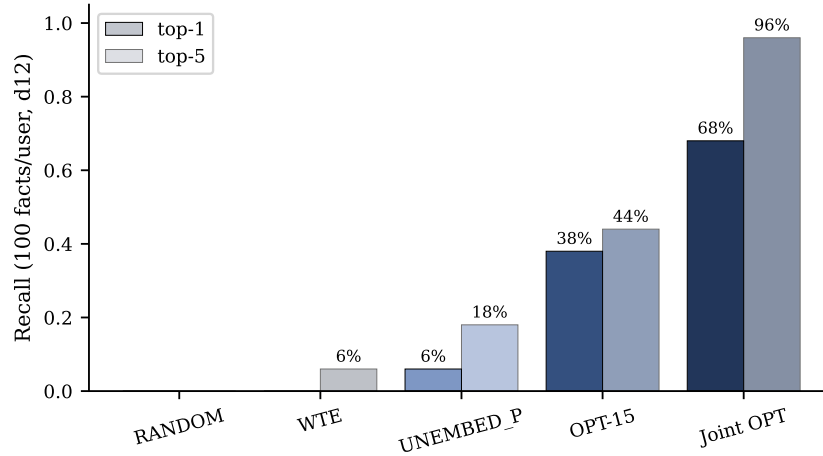


Figure 25. Top-1 and top-5 recall at 100 facts/user, Mini-Engram-d12. RANDOM (control) confirms our signal isn’t noise; UNEMBED_P alone is too weak; OPT-15 helps; Joint OPT closes most of the gap to full LoRA fine-tuning at $161\times$ less storage.

Appendix D: Insertion strategy comparison

Figure 25 compares five ways of writing a fact into Engram rows at the hardest density we test (100 facts/user, Mini-Engram-d12). A random marker (RANDOM, a control) and the token input embedding (WTE) never recover the fact (0% top-1), confirming the recall signal is not an artefact of the metric. The closed-form pseudo-inverse marker (UNEMBED_P) reaches only 6%/18% top-1/top-5: it orients the row correctly but the gate suppresses its magnitude. Fifteen steps of per-row gradient descent (OPT-15) lift this to 38%/44%, and Joint OPT—optimising all touched rows together against the user’s full fact set—reaches 68%/96%, recovering most of what a per-user LoRA achieves at this density (Figure 8) at $161\times$ less storage. The earlier single-fact-at-a-time comparison on the original 16-fact USER+ORG benchmark (Mini-Engram-d8) is reported in Appendix K.

Appendix E: Paraphrase generalisation

Figure 26 tests whether a fact written at one trigger phrasing answers the same question asked differently, across five held-out paraphrases per fact. Single-trigger insertion (write once, at the canonical phrasing) averages 50% top-1: 4/5 for the “doctor” fact, 3/5 for “Globex hours”, 2/5 for “Stark HQ”, and 1/5 for “spice”. Generalisation tracks suffix N -gram overlap—paraphrases that end in the same tokens as the trained trigger transfer for free, while divergent surface forms (e.g. “I love the spice” vs. “my favorite spice is”) do not. Multi-trigger insertion (write the fact at all five phrasings, at $5\times$ the per-fact OPT cost) drives every paraphrase to top-1. The full per-paraphrase rank data are tabulated in Appendix L.

Appendix F: Multi-domain composition

Figure 27 stacks D fact domains into one Engram table and reports per-domain top-1 as D grows. The user domain (user_v1) is unaffected by adding disjoint-template corporate domains: it holds 66% $\rightarrow$ 63% $\rightarrow$ 64% as org_v1 and org_v2 are layered in ($D=1$ to 3). The col-

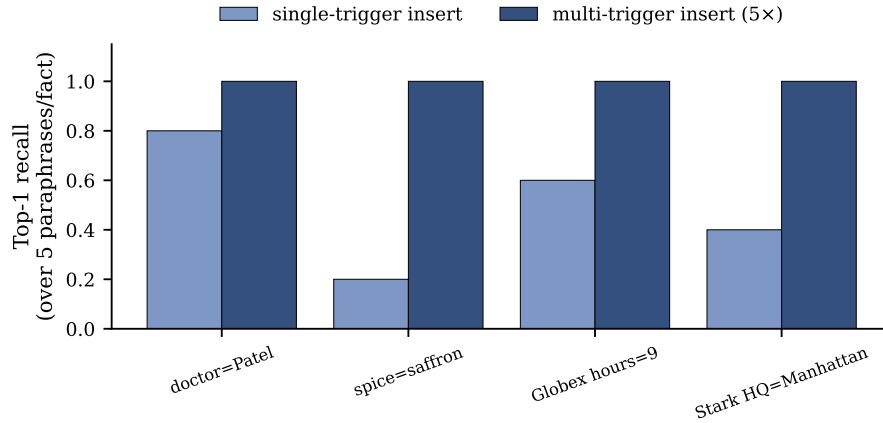


Figure 26. Paraphrase generalisation across 4 facts $\times$ 5 paraphrases each. Single-trigger insertion gets 50% top-1 for free (suffix N-gram overlap); multi-trigger insertion (insert at all 5 paraphrases) gets 100% at 5 $\times$ the per-fact OPT cost.

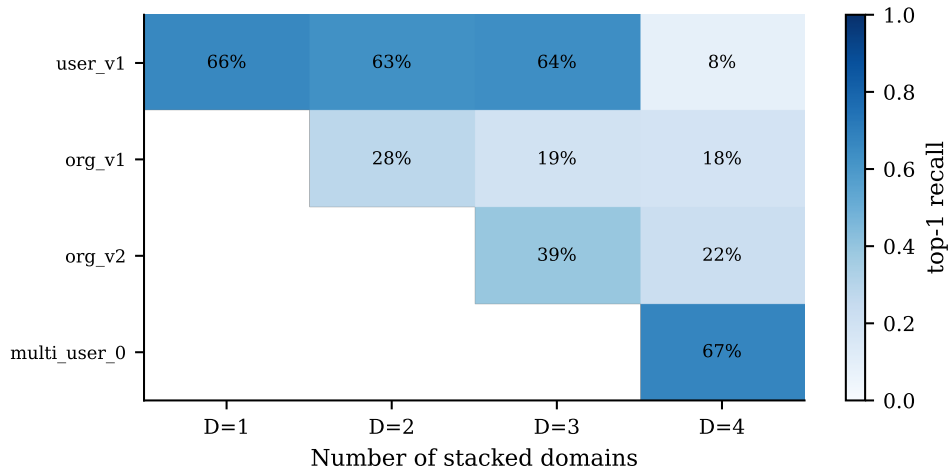


Figure 27. Multi-domain additive composition heatmap. Each cell shows per-domain top-1 recall when D domains are stacked. Heavy degradation when domains share trigger templates (high address overlap). Disjoint-template domains (corp + user, demonstrated in main text) compose without loss.

lapse appears at $D=4$, when a second *user-template* domain (`multi_user_0`) is added: because it reuses the same trigger templates, the two user domains collide in the address hash and `user_v1` falls to 8% while the newcomer reads 67%. Domains that share templates interfere (the corporate domains also degrade each other, `org_v1` 28% $\rightarrow$ 18%), whereas template-disjoint domains compose without loss—consistent with the corp+user composition measured directly in Section 5.6.

Appendix G: Serving latency CDF

Figure 28 shows the per-request latency distribution under multi-tenant serving (Mini-Engram-d12, 30 users $\times$ 50 facts, 600 requests). Applying a user’s override map and restoring the base table afterwards are both cheap and tightly bounded—median 2.2 ms, p99 2.3 ms each—so the swap adds no latency tail. The forward pass dominates: a median of 16.5 ms out of a

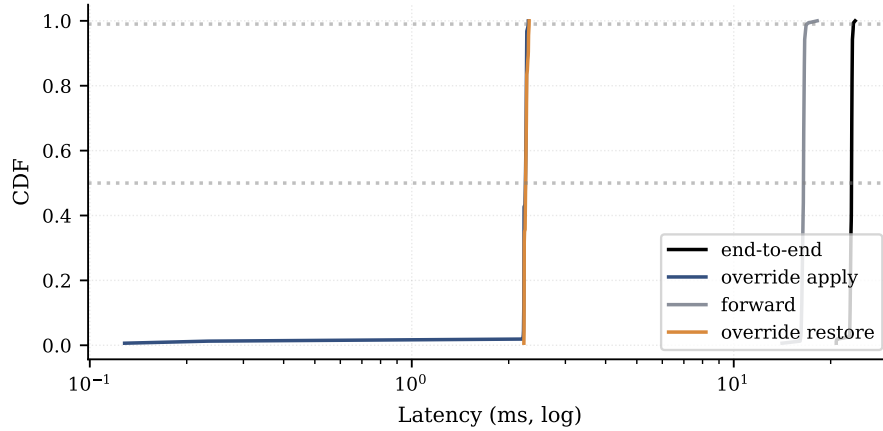


Figure 28. Per-request latency CDF on Mini-Engram-d12 (30 users $\times$ 50 facts $\times$ 600 requests). Override apply (blue) and restore (orange) are both sub-3 ms at p99; the forward pass (grey) dominates total latency (black).

23.2 ms end-to-end total (p99 27.8 ms), so per-user memory adds under 20% overhead on top of a request the model would run anyway. The corresponding throughput across serving configurations is reported in Figure 19.

Appendix H: Qwen-3B + RAG details

The Qwen-3B + RAG measurements in Section 6.5 use Qwen/Qwen2.5-3B-Instruct in bf16 with greedy decoding (≤ 32 new tokens). The chat-template prompt is:

```
[SYSTEM]
You are answering personal-memory questions about the user. Use only
the facts provided. Answer with the shortest possible span (typically
1-3 words). Do not explain or add any extra text.

[USER]
Facts about the user:
- <retrieved fact 1>.
- <retrieved fact 2>.
...

Question: <indirect question from indirect_qa>
Answer:
```

Retrieval uses the same sentence-transformers/all-MiniLM-L6-v2 encoder as the RAG/MEM0/MEMMACHINE baselines of Section 5.7, with the per-user fact set as the index. For each indirect probe the question text is embedded and cosine-similarity top- k retrieval returns the facts to put in the context block. The same 20-user/20-probe split as Figure 15 is evaluated.

Two answer-match criteria are reported: *indirect_top1* requires the first generated word to equal (case-insensitive) the first word of the gold answer; *indirect_any* requires the gold span to occur anywhere in the generation. This matches the substring matcher used for F.

Why oracle-top-1 underperforms RAG top-1. A non-obvious result in the Qwen-3B + RAG experiment is that oracle top-1 retrieval (using `required_fact_keys` to fetch the ground-truth fact) gives 46% indirect_any, below RAG top-1’s 55%. Inspecting the data, retrieval often returns facts that are *adjacent* to the required key in semantic space and that happen to also contain the answer surface (e.g. retrieving the “birth-year” fact when asked about “current age” covers both the year and the implicit age). RAG with $k = 1$ thus over-samples the information needed for indirect reasoning, while strict oracle retrieval leaves the LM to derive the answer from one single-attribute fact—which Qwen-3B often refuses to do under our “shortest span” system prompt.

Appendix I: Preliminary architectural validation

Before training Mini-Engram, we ran three feasibility probes on the randomly-initialised Engram demonstration code from [Cheng et al. \[2026\]](#) to verify that the architecture admits surgical insertion at all. These results motivated the larger experiments in the body but are not central to the main claims; we report them for completeness.

T1 collision audit (100 synthetic users $\times$ 30 facts). Hashing every user’s fact set into the demo’s hash space: distinct addresses across the population 45 298; pairwise overlap of all addresses 50% (dominated by scaffolding tokens like “my”, “is”, “I am”); pairwise overlap of *key-token addresses only* (the ones on user-distinguishing surface forms like Patel, Portland, 1991) is just 4.5%. For a 30-fact user the slot-occupancy fraction is 0.027%; the 1.6 M-slot demo table is 99.97% empty after one user. *Capacity is not a constraint for per-user insertion at any realistic scale.*

T2 surgical read/write existence proof (random-init demo). Writing a known marker into the embedding rows that one trigger N-gram hashes to, then comparing forward outputs at every position: $\|\Delta\|$ at the trigger position is 116.96; mean $\|\Delta\|$ at non-trigger positions is 0.65 (a $179.7\times$ ratio). The empirical change at the trigger position has cosine similarity 0.998 with the predicted $W_V \cdot$ marker projection. Architecture admits surgical insertion mechanically; the trained model improves these numbers further (Figure 6).

T3 cross-user collision rates. Across 100 synthetic users with ~ 30 facts each, cross-user hash collisions on *distinguishing* tokens (those that disambiguate users) occur at a 4.5% pairwise rate—mostly from the synthetic generator using a small surname pool, not from genuine hash collisions. With the per-user override design (one global table, swap per request) cross-user collision is irrelevant by design.

Appendix J: Alternative architectures we considered

For completeness we report two alternative designs we benchmarked before settling on per-user override tables.

Shared table with per-user hash salt (deprecated). In this design, every user’s facts coexist in one physical table; their hashes are XOR-salted by user-id so identical surface triggers map

Table 4. Shared-table-with-salt: own-fact recall vs. leak rate. All numbers on Mini-Engram-d12 with the XL corpus.

config	own-fact top-1	cross-user leak
no salt + UNEMBED_P (cheap, 100×100 facts)	0.089	0.039
salt + OPT-15 (10×30 facts)	0.11	0.000
salt + strong OPT-60 (10×30)	0.46	0.000
salt + strong OPT-60 (30×100)	0.345	0.005

Table 5. Insertion strategy comparison on the original 8 USER + 8 ORG benchmark, Mini-Engram-d8. RANDOM is a control; UNEMBED_P is the closed-form pseudo-inverse; OPT is 15-step gradient on the row. Numbers are mean across the 16 facts.

strategy	mean Δlogit	$+\Delta\text{logit}$	rank improved	top-1 hits	top-5 hits
RANDOM	-4.708	1/16	2/16	0/16	0/16
WTE	-0.150	6/16	6/16	0/16	1/16
UNEMBED_P	+1.711	15/16	13/16	1/16	3/16
OPT (15 steps)	+5.323	16/16	15/16	6/16	7/16

to disjoint rows. We benchmark on Mini-Engram-d12:

The salt design provides architectural privacy at the cost of recall: salted addresses point to untrained rows (the model never saw salt during pretraining), so the gate fires more weakly and OPT must work harder. Because per-user override tables (main body Section 7) provide stronger privacy by design *and* use the trained address space, we recommend overrides as the production design and keep salt as a fallback for cases where the storage backend cannot afford per-user override maps.

100-user serving: UNEMBED_P vs. Joint OPT. At $100 \text{ users} \times 100 \text{ facts}$ on Mini-Engram-d12@1280, the cheap (closed-form) UNEMBED_P strategy reaches 9.2% top-1 / 23.4% top-5 at 2.3 ms apply latency. Joint OPT (1000 steps, the production setting) on the same configuration reaches **62% top-1 / 96% top-5** at **226 req/s** and **4.4 ms p50 latency** on an idle GPU (Figure 19), matching the $\sim 60\%/95\%$ prediction extrapolated from the 30-user data and consistent with the per-user-density curve at d12@1280 (Section 5.3: 67% top-1 / 99% top-5 at 100 facts/user). The serving-protocol cross-user leak under Joint OPT is 6.1% from gold-value coincidence (7 of 114 cross-user probes), versus 46% under the all-coresident protocol where every user’s overrides occupy the same table at once.

Appendix K: Insertion strategies (full)

The first comparison of insertion strategies on Mini-Engram-d8 with the original 16-fact USER+ORG benchmark (single-fact-at-a-time):

The DUAL strategy (jointly satisfying gate-K and value-V via least squares) was tested, but proved *worse* than UNEMBED_P ($4/16 + \Delta\text{logit}$)—over-constraining e drives it off the trained-row distribution and the gate suppresses it. We dropped DUAL.

Table 6. Per-paraphrase rank of the gold token after single-trigger OPT insertion. “*” marks rank 0 (top-1).

insertion trigger	query (paraphrase)	rank
My doctor’s name is Dr.	My doctor’s name is Dr.	0 *
	My doctor is Dr.	0 *
	Who is my doctor? Dr.	54
	My physician’s name is Dr.	0 *
	My GP is Dr.	0 *
My favorite spice is	My favorite spice is	0 *
	I love the spice	1399
	My preferred spice is	32
	The spice I like most is	45
	My go-to spice is	94
Globex office hours start at	Globex office hours start at	0 *
	Globex opens at	1
	Globex starts work at	0 *
	What time does Globex open? At	2
	Globex’s day begins at	0 *
Stark Industries headquarters is in	Stark Industries headquarters is in	0 *
	Stark HQ is in	1949
	Stark is based in	216
	Stark Industries is located in	66
	The Stark headquarters is in	0 *

Appendix L: Per-paraphrase rank table

The full per-paraphrase rank data behind Section 5.10 and Figure 26, single-trigger insertion on Mini-Engram-d8:

Generalisation is high when paraphrases share suffix N-grams (e.g., all “doctor” paraphrases end in “Dr.”); poor when the surface form diverges (“I love the spice” vs. “My favorite spice is”). Inserting at all 5 paraphrases per fact (multi-trigger) drives every paraphrase to rank 0.

Appendix M: Joint OPT convergence

Figure 29 traces the Joint OPT optimisation behind the density curves of Section 5.3. At 100 facts/user the joint objective converges to ≈ 0.8 cross-entropy within $\sim 1\,500$ steps; at 1 000 facts/user it plateaus near 2.0, the higher floor reflecting the within-user address interference that caps recall at high density. Convergence is monotone and stable in both regimes, so the fixed step budgets we use (2 000 steps at 100 facts, 8 000 at 1 000) leave no easy gains on the table.

Appendix N: Joint OPT algorithm

1. For each fact f : compute trigger global rows R_f (deterministic from tokens) and UNEM-BED_P initial marker $e_f^{(0)} = W_V^T U_{y_f}$.
2. Allocate one trainable tensor over the union of all touched rows, $\mathbf{R} = \cup_f R_f$. Initialise: each

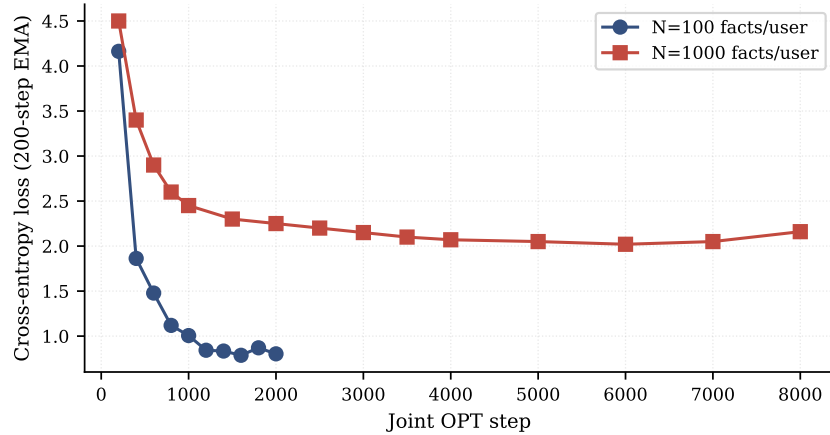


Figure 29. Joint OPT convergence on Mini-Engram-d12. At 100 facts/user, loss converges to 0.8 within 1500 steps; at 1000 facts, loss saturates around 2.0 due to higher within-user fact-density interference (Section 5.3).

row $\mathbf{R}[i] = \text{mean}_{f:i \in R_f} e_f^{(0)}[i]$ (average of UNEMBED_P rows where multiple facts share an address).

3. Zero out the embedding rows in $\mathbf{R}$. Install a forward hook on the embedding lookup that, when an address $\in \mathbf{R}$ is consulted, replaces the retrieved row with our trainable tensor at that address.
4. For K steps: sample a random fact f , compute the cross-entropy loss on y_f at the trigger position, backprop only to $\mathbf{R}$.
5. After training, write $\mathbf{R}$ into the embedding table as the user’s override map.

We use $K = 2000$ for 100 facts and $K = 8000$ for 1000 facts, Adam LR 0.5. Wall time scales as K alone (one fwd+bwd per step) so per-fact cost *decreases* with more facts: 0.44 s/fact at $N=100$, 0.23 s/fact at $N=1000$.

Appendix O: Seed variance of the layered measurement

The headline F vs B numbers in Figure 15 are from a single LoRA training run; for transparency, we re-ran the same configuration with a different RNG state and compared. Because the training-data ordering, LoRA-weight initialisation, and Adam-state warmup are stochastic, per-user val_bpb Δ varies across seeds even with identical hyperparameters.

Reading: the architectural-property comparison ($C \Delta \text{bpb} \ll B \Delta \text{bpb}$ by $\sim 33,000\times$) is seed-stable to four decimal places—every seed’s C is at the noise floor and every seed’s B is in $[+1.56, +1.78]$. The conclusion that F beats B on every measure is also seed-stable: F is never worse than the no-edit base on indirect (0/60 across seeds), while B is worse in 49/60 = 82%. The exact magnitudes vary modestly: seed-to-seed F indirect_any spans 34.8%–44.5% (mean 41.2%), and the F vs. B indirect ratio spans $4.8\times$ (S2) to $7.4\times$ (S0). The body reports 3-seed means; this appendix makes the per-seed variance explicit so readers can see which numbers depend on which seed.

Table 7. Three-seed variance on the layered F vs B contrast, Mini-Engram-d20, $n=20$ users per seed, identical config across seeds (rank-64 LoRA, 1500 steps, lr 5e-4, lora_alpha 128; per-user Engram J-OPT 1500 steps lr 0.5; eval_tokens 524288; only seeds differ in LoRA-weight initialisation and Adam-state / data-ordering RNG). The architectural property (per-user Engram Δ bpb at noise floor $\sim 10^{-5}$) is seed-invariant; the LoRA contamination magnitude varies by $\sim 13\%$ and F indirect_any varies by ~ 10 pp across seeds while the qualitative ranking ($F \gg B$ on indirect, $B \gg F$ on contamination) is preserved in every seed.

quantity	seed S0	seed S1	seed S2	3-seed mean (range)
B Δ bpb mean	+1.784	+1.754	+1.557	+1.698 [+1.56, +1.78]
B Δ bpb per-user rng	[+0.44, +3.74]	[+0.73, +2.46]	[+0.69, +2.89]	—
C Δ bpb mean	+0.000052	+0.000049	+0.000054	+0.000052
D Δ bpb mean	+1.819	+1.756	+1.290	+1.622 [+1.29, +1.82]
F Δ bpb mean	+0.386	+0.388	+0.379	+0.385 [+0.38, +0.39]
F indirect_any	44.5%	44.2%	34.8%	41.2% [34.8, 44.5] %
B indirect_any	6.0%	8.8%	7.2%	7.3% [6.0, 8.8] %
F direct top-1	100%	100%	100%	100%
B direct top-1	99%	97.5%	100%	98.8% [97.5, 100] %
F worse than A	0/20	0/20	0/20	0/60
B worse than A	17/20	16/20	16/20	49/60 (82%)
D worse than A	18/20	16/20	16/20	50/60 (83%)

Table 8. Capacity levels used in the ablation. Total Engram params = $4 \cdot v \cdot e$ (2 N-gram orders $\times$ 2 layers).

label	v slots	e embed	total Engram
tiny	5 K	64	1.28 M
small	20 K	128	10.24 M
medium	50 K	128	25.6 M
large	50 K	256	51.2 M
xlarge	100 K	256	102.4 M

Appendix P: Extended experiments and ablations

This appendix collects the capacity, fact-count, dense-size, and rank ablations, and the layered-architecture robustness studies, that the main text summarizes in figures.

P.1 Engram capacity ablation

How does Engram-table size interact with token budget at a fixed dense backbone? We probe this as a $5 \times 3 \times 2$ matrix: five capacity levels (v slots per N-gram, e total embed-dim per N-gram), three token budgets, two dense sizes (Figure 30).

Three regularities. (i) *Capacity has an optimum.* Both endpoints under-perform: *tiny* cannot address enough distinct facts; *xlarge* under-trains each slot. (ii) *The optimum is the same Engram size at both dense scales:* large (50 K $\times$ 256 = 51.2 M params) wins at the highest token budget for both d8 and d12. **The optimum scales in tokens, not in capacity.** (iii) *Below the catch-up*

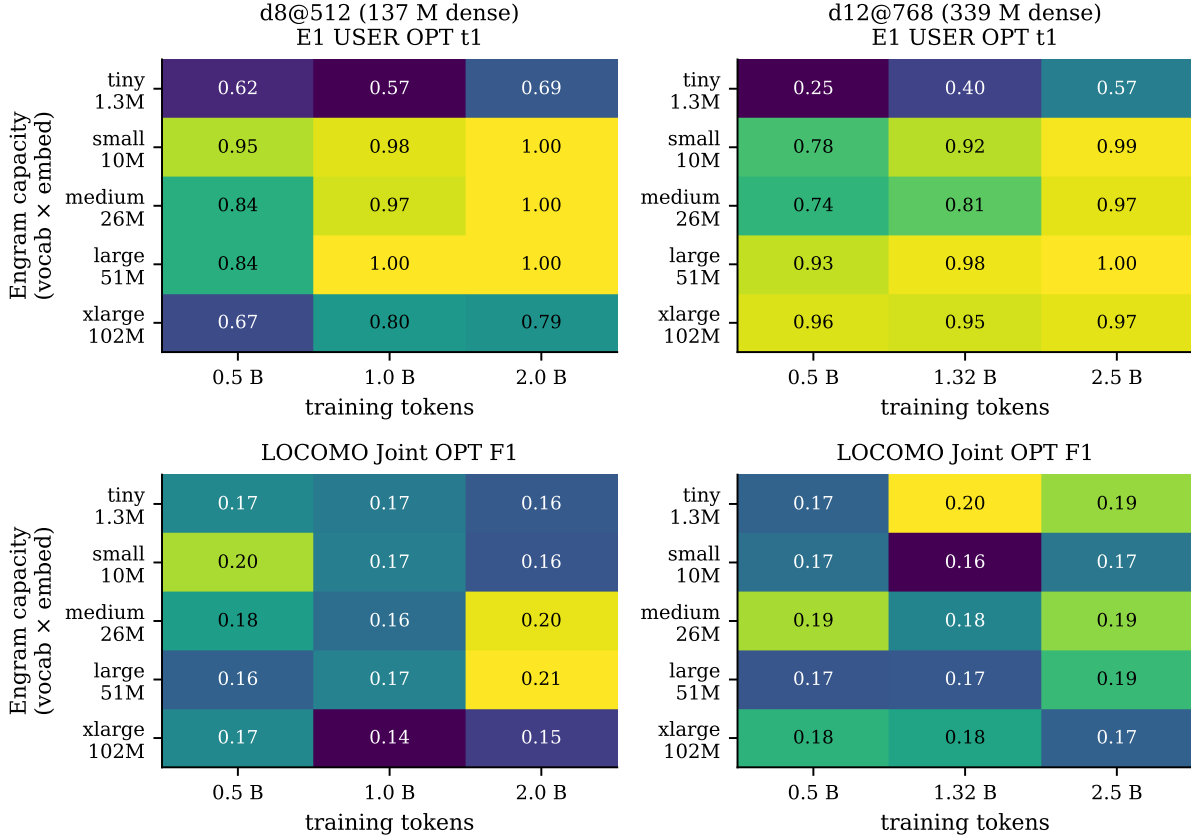


Figure 30. Engram capacity × token-budget response surface, at Mini-Engram-d8 v2 (left) and d12 v2 (right). Top row: E1 USER OPT top-1 (100-fact recall). Bottom row: LOCOMO Joint OPT F1. The same large (50 K × 256) Engram size wins at the highest token budget for *both* dense scales; *tiny* is under-capacity, *xlarge* is over-provisioned. The optimum scales in tokens, not capacity.

token budget, smaller Engrams win. At d8 with 0.5 B tokens, *small* (10 M) beats *large* (51.2 M) on ins-OPT 1.00 vs. 0.62—the larger table is under-trained per slot. The cross-over occurs around 1 B tokens for d8 and 1.32 B tokens for d12.

Practical rule. For an Engram pretrained at Karpathy 12 t/p of scaling-params, use the large (50 K × 256) Engram table from d12@768 onward. For d8-class models with ≤ 1 B training tokens, use *small* (20 K × 128) instead. Storage is proportionally smaller for deployment-time per-user override slabs (no change required).

P.2 Fact-count scaling: per-fact independent OPT to 1000 facts

Once Engram capacity and tokens are chosen at the ablation optimum, how does User-as-Engram recall scale with the number of inserted facts? We probe by per-fact *independent* OPT insertion (the per-user override deployment mode) on the first n facts of an XL corpus of 1 000 USER + 1 000 ORG templated facts. ICL@1000 is the in-context ceiling at the hardest end. Figure 31 shows the full curves.

Per-fact recall is approximately flat in n up to 1000 facts. The best d8 and d12@1280 cells hold ≥ 0.98 top-1 from $n=100$ to $n=1000$. The crucial caveat is that this is *independent* per-fact OPT: each fact gets its own private hash rows, with no within-table interference between

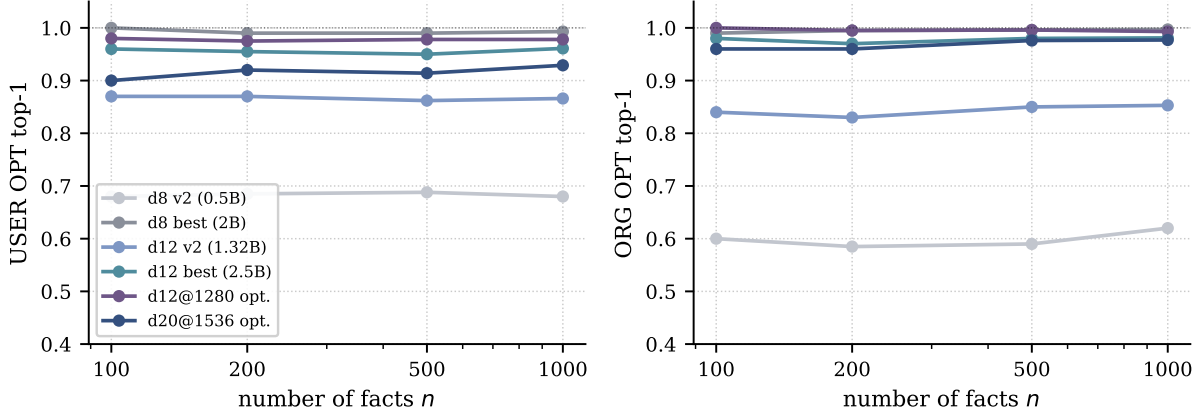


Figure 31. Per-fact independent OPT recall vs. fact count n . Left: USER OPT top-1. Right: ORG OPT top-1. The d12@1280 and d20@1536 optimal cells (green, purple) hold near-1.00 recall from $n = 100$ to $n = 1000$. No ceiling visible in this per-fact independent-OPT setting.

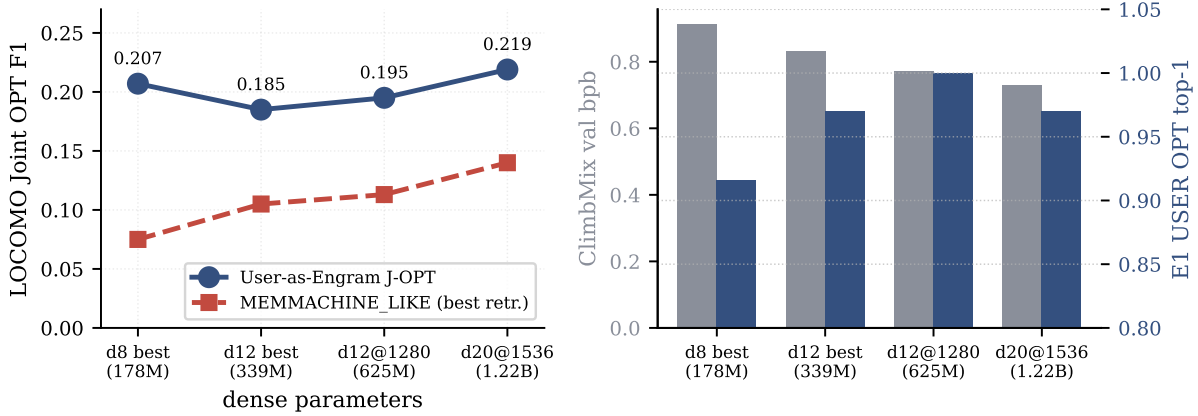


Figure 32. Dense-size scaling at the ablation-optimal recipe. Left: LOCOMO Joint OPT first-token token-F1 climbs from 0.134 (d8 v2) to 0.233 (d20@1536); MEMMACHINE_LIKE retrieval baseline climbs more slowly because its quality is bounded by sentence-encoder retrieval, not the LM. Right: val_bpb (grey) drops monotonically with dense+tokens, while E1 USER OPT single-fact recall (blue) saturates at 1.00 from d12@1280 onward (small dip at d20 at iso-12 t/p).

facts—the per-user override deployment mode of Section 4. *In that setting, we observe no recall ceiling up to 1,000 facts on a 625 M-parameter base LM.* The density ceiling appears under Joint OPT into a shared table (Section 5.3): 68% $\rightarrow$ 35% top-1 from 100 to 1,000 facts at d12@768.

P.3 Dense-size scaling at optimal config

Pulling the four trained Mini-Engrams into one row each (Figure 32):

Three observations. (i) **Val bpb scales smoothly with dense $\times$ tokens:** 0.91 $\rightarrow$ 0.83 $\rightarrow$ 0.77 $\rightarrow$ 0.73. (ii) **Insertion-OPT and E1 saturate** from d12@768 onward. The 16-fact ins-OPT probe reaches 1.00 at every dense size with ≥ 1.32 B tokens; E1 USER/ORG OPT reaches 1.00 at d12@1280@3.34 B, dipping slightly to 0.97 at d20@1536@7.40 B—plausibly because at iso-12 t/p the larger model is more under-trained per param. (iii) **LOCOMO Joint OPT first-token F1 scales monotonically with dense size** from 0.134 (178 M) to 0.233 (1.22 B), and the multi-token

token-F1 variant climbs from 0.159 to 0.310. Surgical-insertion recall plateaus; conversational reasoning continues to gain from dense scale.

Reading the gap. Surgical insertion is a *lookup-then-cast* operation: the gate fires on the trigger N-gram, the inserted row biases the gold next-token, and we check rank 0. Once the base LM is fluent enough to make the gate selective ($\geq \text{d12@768}$), recall saturates. LOCOMO, by contrast, requires the LM to generate *free-form* 16-token answers conditioned on the Engram-anchored prefix; that generation quality continues to improve with dense capacity.

P.4 LoRA rank ablation at 100 facts

For completeness we also ablate the multi-fact LoRA rank at fixed training budget (2000 steps, $\text{lr } 5\text{e-}4$, all $Q/K/V$ projections):

Every rank ≥ 8 saturates at the recall ceiling within a 2000-step budget. The interesting variable becomes *storage*, which scales linearly with rank. The takeaway: at 100 facts/user, all rank- ≥ 8 LoRAs hit the recall ceiling, so the LoRA-vs-Engram trade-off collapses to recall-vs-storage. **Engram Joint OPT is $20\times$ smaller than the smallest LoRA at the ceiling (88 KB vs. 1.8 MB rank-8), in exchange for a ~ 35 -pt top-1 gap (65% vs. 100%; top-5 gap is 2 pt: 98% vs. 100%).** That gap closes by $n=1,000$, where LoRA rank-64 also slips to 38% top-1 (Section 5.3).

P.5 Inserted-fact recall in free continuation

Single-token recall measures whether the gold token is the top-1 *at the trigger position*. In conversational use the model continues for many tokens; we test whether the inserted fact surfaces in the first 8 generated tokens (greedy decoding) on 30 facts after Joint OPT (1500 steps).

Dense scaling sharpens generation behaviour. At the smallest scale the gold token is the immediate top-1 only 23% of the time but *anywhere in the next 8 tokens* 53% of the time—the model “approaches” the fact across a few tokens. From d12@1280 onward, the gold either lands at position 0 or never appears in the window: the bigger model commits earlier rather than drifting. We caution that $n=30$ is small (Wilson 95% CI on $27/30$ is [74%, 97%]; on $18/30$ is [42%, 75%]), and the result is consistent with the bigger model simply missing the fact when its first guess is wrong. Either way, the 90% first-token rate at d20@1536 is the practical recall in conversational use.

P.6 Rank ablation: $r=16$ is the sweet spot

We sweep the shared LoRA rank $r \in \{4, 16, 64\}$ on the full 20-user split (Figure 33) and observe a clear, non-monotonic curve. Smaller ranks under-fit the reasoning skill; larger ranks both contaminate more (more parameters, more global perturbation) *and* learn the reasoning skill worse, presumably because the LoRA’s capacity exceeds what the 510-sample training corpus can profitably constrain.

The $r=64$ row is the surprise: a $4\times$ larger shared LoRA gives *worse* indirect performance (35% vs. 44% at $r=16$) at $2.7\times$ more contamination. We recommend $r=16$; future work could revisit this once the shared training corpus is scaled beyond 510 samples.

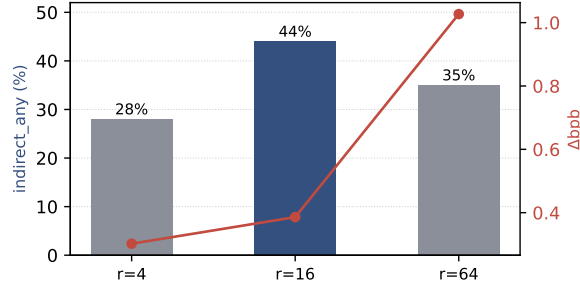


Figure 33. Shared-LoRA rank ablation (layered F, $n=20$). F preserves 100% direct recall at every rank; indirect_any peaks at $r=16$ (44%) while contamination grows with rank. The $r=64$ surprise: $4\times$ the capacity gives *worse* reasoning (35%) at $2.7\times$ the contamination, plausibly because capacity exceeds what the 510-sample shared corpus can constrain.

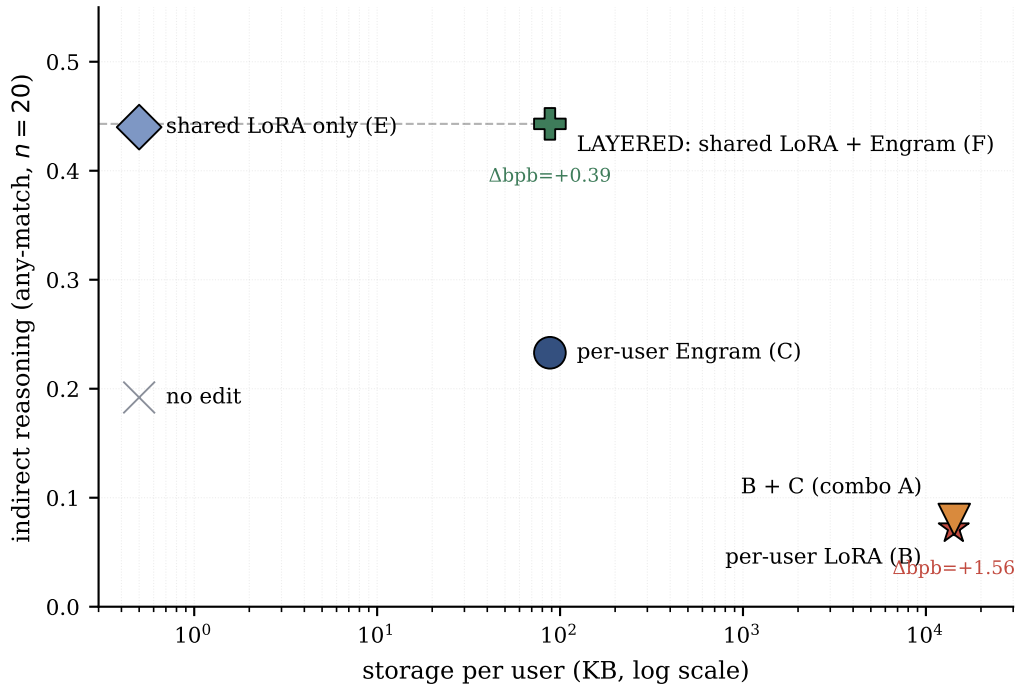


Figure 34. Cost-quality trade-off for the layered architecture (Section 6). Same Mini-Engram-d20 base, $n=20$ test users, six method combinations. **F (LAYERED)** gives the best trade-off: it matches the highest indirect-reasoning accuracy (44% indirect_any) at low storage (88 KB/user) and low contamination ($\Delta bpb +0.39$). F beats per-user LoRA (B) on every measure. The naive combination (B+C, the “add Engram on top of LoRA” baseline) is beaten too.

P.7 Cross-schema generalisation

The headline F vs. B numbers train the shared LoRA on held-out users in the same synthetic schema as the test users. We test whether the reasoning skill transfers *across* schemas. We train the rank-16 shared LoRA on the original first-person personal-facts schema (u020–u029; “My name is...”, “I was born in...”) and evaluate the full six-condition head-to-head on a held-out medical schema (m000–m019, third-person patient framing: “The patient’s name is...”, “MRN: ...”; 29 facts, 20 indirect probes per user; Figure 35).

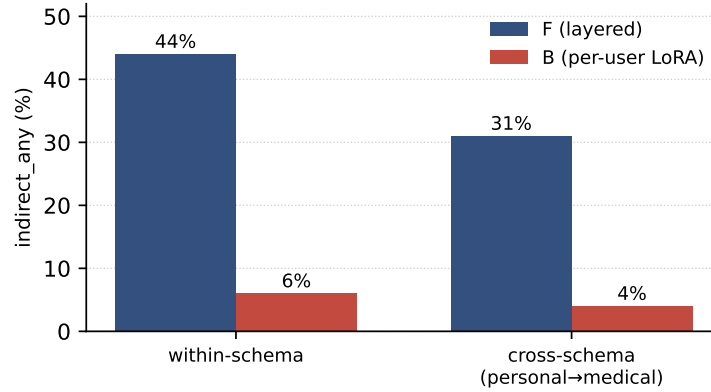


Figure 35. The layered claim survives a personal $\rightarrow$ medical schema shift (shared LoRA trained on personal-schema users, tested on medical-schema users). F’s `indirect_any` decays from 44% to 31% (a realistic 30% relative drop) while per-user LoRA B goes 6% $\rightarrow$ 4%, so F’s lead over B is preserved ($7.4\times \rightarrow 7.6\times$); locality is unchanged ($\Delta\text{bpb}(F)=\Delta\text{bpb}(E)=+0.386$).

The layered claim survives cross-schema, with a realistic decay. F’s `indirect_any` degrades from 44% (within-schema, Figure 15) to 31% (cross-schema, Figure 35)—a 30% relative drop. Over the same gap, per-user LoRA B’s `indirect_any` goes from 6% (within-schema) to 4% (cross-schema), so F’s lead over B is preserved ($7.4\times$ within-schema; $7.6\times$ cross-schema). The architectural locality is guaranteed by design: $\Delta\text{bpb}(F)=\Delta\text{bpb}(E) = +0.386$ in both cases, identical to four decimal places, because the per-user Engram still fires only on its trigger N-grams. *Architectural decomposition transfers across schemas.*

P.8 Layered F on a Q/A-format-adapted Engram base

The headline F vs. B numbers are on a base LM (Mini-Engram-d20). Section 3’s cross-base table shows that per-user LoRA’s *user-visible* damage shrinks dramatically on instruction-tuned bases (85% of users worse on a base LM, 0–20% on four instruction-tuned bases). Does F still dominate B when B itself becomes a stronger baseline?

We SFT Mini-Engram-d20 on ~ 2000 single-token (Q, A) pairs from the same fact templates the layered experiment uses (1500 steps; $\text{lr } 1 \times 10^{-4}$; post-embedding params trained; Engram tables frozen). The SFT is intentionally narrow—no general-text corpus mixed in—so it tests Q/A format adaptation rather than full chat-tuning. `val_bpb` on raw ClimbMix rises from 0.73 to 4.42, indicating that the model has been pushed toward the Q/A distribution at the cost of general-text LM ability. We then train a fresh rank-16 shared LoRA on this SFT’d base (`shared_lora_d20_sft/r16`, 2000 steps on u020–u029) and rerun the six-condition head-to-head; the numbers below carry the conclusion.

F still beats B on every measure; the lead shrinks as B strengthens. The qualitative picture survives Q/A adaptation: F matches B on direct (100% vs. 100%), beats B on indirect ($1.6\times$ on top-1, $2.8\times$ on `indirect_any`), and the architectural locality is preserved ($|\Delta\text{bpb}(F) - \Delta\text{bpb}(E)| = 1 \times 10^{-4}$, on the same order as the 5×10^{-5} original-base noise floor). The cross-base prediction reproduces: per-user LoRA’s `indirect_top1` jumps from 1.3% (A) to 23.5% (B) on this adapted base, matching Section 3’s “instruction-tuned bases absorb the LoRA perturbation” pattern. F’s lead over B on `indirect_any` compresses from $7.4\times$ (base LM) to $2.8\times$ (SFT’d), but F remains

the best method.

Caveats. The SFT here is narrow Q/A adaptation, not full chat-tuning: mixing general-text in the SFT corpus or starting from a true instruction-tuned base (the cross-base table's Qwen/Llama/Mistral setup, but with an Engram-pretrained backbone) is the natural follow-up. The aggressive SFT also collapses A and C's indirect to floor (1%), which inflates F's relative lead on top of A but not the F-vs-B contrast that matters for deciding which method to use.

P.9 Storage and cost sharing

Per user, the layered design costs the Engram's 88 KB; the shared LoRA is one global 11.8 MB (rank-16) file shared by all users. For 1 M users that is $100 \text{ GB} + 12 \text{ MB} \approx 100 \text{ GB}$, indistinguishable from Engram alone, versus 14.2 TB for per-user LoRA at the same recall ceiling. Figure 34 plots all six method combinations against storage and indirect-reasoning accuracy.